UriNet

Build the simplest modular blockchain for real-life utility: a network of plug-and-play, single-purpose chains—natively interconnected and anchored by a common core: UriChain—sharing universal payments, built-in security, and live data so products just work.

Ryan Kwak ryankwak@uriverse.com

Jun Hyuk Kwak jkwak@uriverse.com

Boram Kim boram@uriverse.com

NOTE: This document and project is under active development and everything is subject to change without notice—see disclaimer below.

NOTICE AND DISCLAIMER

PLEASE READ THIS ENTIRE NOTICE AND DISCLAIMER CAREFULLY. NOTHING HEREIN CONSTITUTES LEGAL, FINANCIAL, BUSINESS, ACCOUNTING, OR TAX ADVICE. YOU SHOULD CONSULT YOUR OWN INDEPENDENT LEGAL, FINANCIAL, TAX, OR OTHER PROFESSIONAL ADVISORS BEFORE ENGAGING IN ANY ACTIVITY RELATED TO THIS DOCUMENT, THE URI TOKEN, OR THE URINET NETWORK.

Neither the project's sponsoring entity or entities (together, the Organization), any developers, core contributors, advisors, contractors, or service providers (together, the Team), nor any distributor or vendor of URI (each, a Distributor) shall be liable for any direct or indirect losses, damages, or claims arising from or related to this Whitepaper, any related websites, repositories, or materials (collectively, the Materials).

Project purpose

The project seeks to research, design, build, and promote a utility-focused blockchain ecosystem (UriNet) intended to support real-world applications and improve everyday life. UriNet's native token, URI, is intended solely as a functional, cryptographically secured utility token used within the UriNet ecosystem. The Organization may support research, development, community programs, and operations that further this purpose.

Nature of this Whitepaper

This Whitepaper is provided strictly for general informational and discussion purposes only. It is not a prospectus, offer document, advertisement, or solicitation for investment, and does not constitute an offer to sell, a solicitation to buy, or a recommendation regarding any securities, financial instruments, or other regulated products in any jurisdiction. The Materials do not and are not intended to create any contractual or other legal rights between you and the Organization, the Team, any Distributor, or any affiliate thereof.

The information herein may be incomplete, inaccurate, or outdated. Assumptions, architectural descriptions, token functionality, roadmaps, timelines, and other content are preliminary, illustrative, and subject to change, suspension, delay, or abandonment at any time, at the Organization's sole discretion, without notice. No representation, warranty, or undertaking (express or implied) is made as to the accuracy, completeness, reliability, merchantability, fitness for a particular purpose, non-infringement, or timeliness of the Materials.

If the Materials contain information sourced from third parties, the Organization and the Team have not independently verified such information and make no representation or warranty regarding it.

No offer; definitive terms control

Nothing in the Materials constitutes, or should be construed as, an offer to sell or the solicitation of an offer to buy URI or any other asset. Any sale, distribution, exchange, or transfer of URI (if any) will occur, if at all, only pursuant to definitive documentation governing such transaction (e.g., token sale terms, token purchase agreement, acceptable use policy, privacy policy, or network terms, collectively, the Definitive Terms). The Definitive Terms, if and when provided, will control in the event of any inconsistency with this Whitepaper. No person is obligated to enter into any contract or binding legal commitment relating to URI based on the Materials.

Eligibility; compliance; sanctions

Access to URI or participation in UriNet may be restricted or prohibited for certain persons or in certain jurisdictions. Without limiting the foregoing, you represent and warrant that: (a) you will ensure at your own expense full compliance with all laws, regulations, and restrictions applicable to you; (b) you are not a citizen, resident (tax or otherwise), domiciliary, or green-card holder of, located in, or otherwise ordinarily resident in any jurisdiction where token sales, crypto-asset activities, or participation in similar projects are restricted or prohibited, or where such participation would require registration or licensing not obtained; (c) you are not a person or entity on, or controlled by persons on, any sanctions or restricted-party lists, and you are not subject to export-control restrictions; and (d) you are of legal age and have the full capacity to participate.

Deemed acknowledgments, representations, and warranties

By accessing or relying on any part of the Materials, you are deemed to acknowledge, represent, and warrant to the Organization, any Distributor, the Team, and their affiliates that:

1. You will not rely on the Materials in making any purchase, sale, investment, or other decision; you will rely only on your own advisors and, where applicable, the Definitive Terms.

2. URI may have no monetary value; there is no guarantee or representation of liquidity, transferability, price, demand, or market for URI.

3. URI is intended solely to enable usage of, and interaction within, UriNet as described (and as may be changed); URI is not designed for speculative investment, profit expectations, or passive income.

4. None of the Organization, the Team, any Distributor, nor their affiliates are responsible for the value, transferability, or availability of any market for URI, whether on third-party exchanges or otherwise.

5. You understand and accept the risks described in the Risk Factors section below and any additional risks that may be set out in other project documentation.

Token characterization; no rights

URI is intended to be a non-refundable, functional, utility token used within UriNet. URI does not represent or confer: (i) any equity, ownership, shareholding, security, debt, or equivalent rights; (ii) any rights to dividends, distributions, profits, interest, or other returns; (iii) any governance, voting, or management rights in the Organization or its affiliates (except, if implemented, on-protocol voting mechanisms that may affect certain technical parameters of UriNet and which may be modified, curtailed, or discontinued at any time); (iv) any claim, proprietary interest, or entitlement to intellectual property; or (v) any right to receive financial statements or other corporate information.

URI is not money, legal tender, electronic money, negotiable instrument, deposit, investment contract, commodity, derivative, or any other regulated financial product. URI is not a loan to the Organization or any other person.

Token functionality (illustrative only; subject to change)

Subject to implementation and change, URI may be used to (a) pay for network usage (e.g., transaction execution, bandwidth, storage, or other metered resources), (b) participate in protocol-level mechanisms (e.g., staking or delegation to support network security, if implemented), and (c) access or interact with applications, services, or modules deployed on UriNet. Any rewards or incentives (if any) are not

guaranteed, may be modified or terminated, and may be subject to additional eligibility, lockups, slashing, or clawback conditions.

Secondary markets; independent exchanges

To the extent a secondary market for URI develops, it is operated by independent third parties. The Organization and the Team do not operate, control, or endorse any exchange or marketplace, provide any listing, market-making, or liquidity support, or make any representations about availability or legality of trading URI in any jurisdiction.

Informational purposes only; no reliance; no duty to update

The Materials describe potential features and a high-level technical and product direction. They are conceptual and may change substantially. Do not rely on the Materials for purchasing, investment, accounting, or tax decisions. The Organization has no obligation to update, correct, or provide continued access to the Materials.

Regulatory status; approvals

No regulator has examined, approved, or endorsed the Materials or URI. Publication, distribution, or dissemination of the Materials does not imply compliance with applicable laws, rules, or regulations.

Forward-looking statements

The Materials may include forward-looking statements, including statements about plans, intent, belief or current expectations, objectives, projections, future events, or performance. These statements are subject to numerous known and unknown risks and uncertainties that could cause actual results to differ materially. You should not place undue reliance on forward-looking statements. The Organization undertakes no obligation to update such statements.

Risk factors (summary, non-exhaustive)

Participating in UriNet or acquiring/holding/using URI involves significant risks, including but not limited to: (a) protocol, software, or smart-contract defects; (b) cryptographic advances or attacks; (c) validator

or network failures, congestion, forks, governance disputes, or chain halts; (d) cybersecurity incidents, key loss, phishing, and operational errors; (e) volatility, illiquidity, or loss of value; (f) regulatory, tax, and compliance changes or enforcement; (g) market, economic, or business risks; (h) reliance on third-party infrastructure (wallets, exchanges, oracles); (i) adverse outcomes from upgrades, parameter changes, or governance actions; and (j) force majeure events. This summary is not exhaustive; additional risks may be described in other documentation, including the Definitive Terms.

Taxes

You are solely responsible for determining, reporting, and paying any taxes arising from your activities related to UriNet or URI, and for maintaining all required records. The Organization does not provide tax advice.

No fiduciary duties; no advice

The Organization and the Team owe you no fiduciary duties, and no communication or information constitutes investment, legal, accounting, or tax advice. You should obtain independent professional advice as needed.

Intellectual property; third-party references

All trademarks, service marks, trade names, logos, and content are the property of their respective owners. References to third-party projects, companies, or platforms are for illustrative purposes only and do not imply affiliation, endorsement, or sponsorship.

Language; translations

This Whitepaper may be translated. The English version controls in the event of any conflict or ambiguity. By accessing the Materials, you acknowledge that you have read and understood the English version.

Privacy; KYC/AML

Any collection or processing of personal data (including for KYC/AML/sanctions screening, if applicable) will be conducted pursuant to the project's privacy policy and applicable law. Participation may require

you to provide information and documentation for compliance purposes; failure to do so may result in denial or termination of access.

No distribution; restrictions on use

No part of the Materials may be copied, reproduced, redistributed, or disseminated, in whole or in part, without the prior written consent of the Organization. Access to the Materials may be restricted in certain jurisdictions. You are responsible for ensuring lawful access.

Limitation of liability; disclaimer of warranties

TO THE MAXIMUM EXTENT PERMITTED BY LAW, THE MATERIALS AND ANY SOFTWARE OR DOCUMENTATION ARE PROVIDED “AS IS” AND “AS AVAILABLE,” WITHOUT WARRANTIES OF ANY KIND, WHETHER EXPRESS, IMPLIED, OR STATUTORY. TO THE MAXIMUM EXTENT PERMITTED BY LAW, IN NO EVENT SHALL THE ORGANIZATION, THE TEAM, ANY DISTRIBUTOR, OR THEIR AFFILIATES BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, EXEMPLARY, OR PUNITIVE DAMAGES, OR FOR ANY LOSS OF PROFITS, REVENUE, DATA, GOODWILL, OR USE, ARISING OUT OF OR RELATED TO THE MATERIALS, URINET, URI, OR ANY ACTIVITIES CONTEMPLATED HEREIN, WHETHER IN CONTRACT, TORT (INCLUDING NEGLIGENCE), STRICT LIABILITY, OR OTHERWISE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Plain-English summary (non-binding; convenience only)

This is a technical, exploratory document about a utility network (UriNet) and a utility token (URI) built to support real-life applications. It’s not investment advice, not an offer to sell anything, and not a promise that anything will ship as described. Everything here may change. If we ever sell or distribute tokens, it will be under separate, definitive terms. Using crypto is risky; do your own research and talk to qualified professionals.

Abstract

UriNet is a modular blockchain network designed for real-life utility applications and AI-driven agentic workflows that require both dedicated execution and native coordination across specialized services. Its core purpose is to provide a shared trusted environment in which off-chain AI agents, users, and service providers can collaborate, transact, settle obligations, submit evidence, and resolve disputes without having to rely on a single central operator or trust one another by default. Rather than forcing unrelated products and services onto one general-purpose chain, UriNet is organized around UriChain as a shared core, a set of sovereign single-purpose Utility Chains, and common ecosystem infrastructure through UriPortal and the Inter-Chain Message System (ICMS). This architecture allows specialized chains to remain narrowly scoped and operationally independent while still using shared rails for coordination, settlement, discovery, interoperability, and service composition. In this model, AI inference and execution can remain off-chain where appropriate, while the trust-critical parts of real-world workflows such as permissions, commitments, evidence, settlement, and final outcomes are coordinated through the network in a structured and verifiable way. ICMS provides the standardized coordination layer for both direct interchain delivery and multi-chain workflows, while URI serves as the common utility asset for fees, staking, governance participation, and shared-service usage across the network.

Introduction

Many useful real-world blockchain applications do not fit cleanly into a single-chain model. Services such as tokenization, escrow, settlement, oracle-driven execution, insurance,

advertising, and other composite products often require separate execution environments, different operating assumptions, and reliable coordination across multiple domains.

The same is also increasingly true for AI-agent and mixed human-agent workflows, where multiple systems may need to coordinate around authority, evidence, approvals, settlement conditions, and shared execution state.

UriNet is built around these premises. Instead of treating interoperability as an afterthought, it defines a coordinated network in which UriChain anchors shared economics, governance, and core trust primitives, Utility Chains execute product-specific logic, UriPortal provides ecosystem access and discovery, and ICMS enables standardized interchain communication and multichain execution flows. This same architecture is also intended to support off-chain AI agents, users, and service providers as they coordinate across real-world workflows that require verifiable permissions, evidence, settlement, and dispute handling. The result is an application-first network intended to let specialized chains operate with clear boundaries while still participating in one coherent system with shared infrastructure, predictable URI-based economics, and a common framework for composing services across the ecosystem, including services that support safe, transparent, and verifiable agentic coordination.

What’s UriNet?

UriNet is the world’s first blockchain that provides the tools needed so that AI agents existing outside the blockchain can communicate with the real world, collaborate with one another, use one another to accomplish their goals, and reliably provide the many kinds of services the market needs.

It is designed as a modular utility network organized around a shared mainnet, **UriChain**, and a set of sovereign, single-purpose **Utility Chains**. Instead of forcing every product into one general-purpose blockchain, UriNet lets each application run on its own chain while still participating in a common network for payments, interoperability, discovery, and shared services. This preserves application-level control without isolating each service into its own disconnected system.

In the original UriNet model, interoperability was described broadly as native interchain messaging. In the current architecture, that idea is made more concrete through the **Inter-Chain Message System (ICMS)**: a network-level messaging and coordination framework that allows chains to exchange instructions, attestations, and asset-related messages in a standardized way. ICMS is designed to support both simple chain-to-chain delivery and more advanced multi-chain workflows in which one application depends on data or actions from several other chains before final execution. This is what allows UriNet to support not only independent utility chains, but also **multichain-dependent services** such as insurance, escrow, tokenization, oracle-driven execution, and other composite applications.

UriNet therefore should be understood not as one blockchain with add-ons, but as a coordinated application network. **UriChain** provides the common economic and governance base. **Utility**

Chains execute product-specific logic. **UriPortal** provides the ecosystem gateway for discovery, onboarding, registration, and standard service access. **ICMS** provides the coordination layer that lets those chains work together as one system. **URI**, the network's native utility token, serves as the common unit for access, fees, staking, and shared service usage across the network.

UriNet is also designed to **natively support AI-driven agentic workflow**. As AI agentic automation becomes more prevalent, it is important to ensure that we not only have the tools to support it, but that the underlying coordination, permissions, evidence, and outcomes can be expressed in a structured and verifiable way. The goal is not to force inference itself on-chain, but to give off-chain agents a trustworthy network environment in which they can operate safely, transparently, and under clear rules.

Some features include:

- **Native support for MCP-style and tool-based environments.** This allows agents to interact with UriNet services through clearer, bounded interfaces rather than arbitrary integrations.
- **Protected work lanes.** The goal is not to encourage blind automation but to streamline safe, automated interactions. UriNet can help agents operate against explicit permissions, delegated authority, structured interactions, and verifiable on-chain outcomes instead of arbitrary connections.
- **Shared trust primitives for agent coordination.** As agentic systems become more capable, they need common rails for commitments, approvals, evidence, settlement, and workflow state. UriNet is designed to provide these functions at the network level so multi-party and multi-agent activity can be coordinated more safely.

- **Composable access to network services.** Because UriNet services are distributed across specialized chains and modules, UriNet and its components can serve as a coordination layer that makes those services easier for AI agents to discover, call, and combine into larger multi-step workflows.

Core components

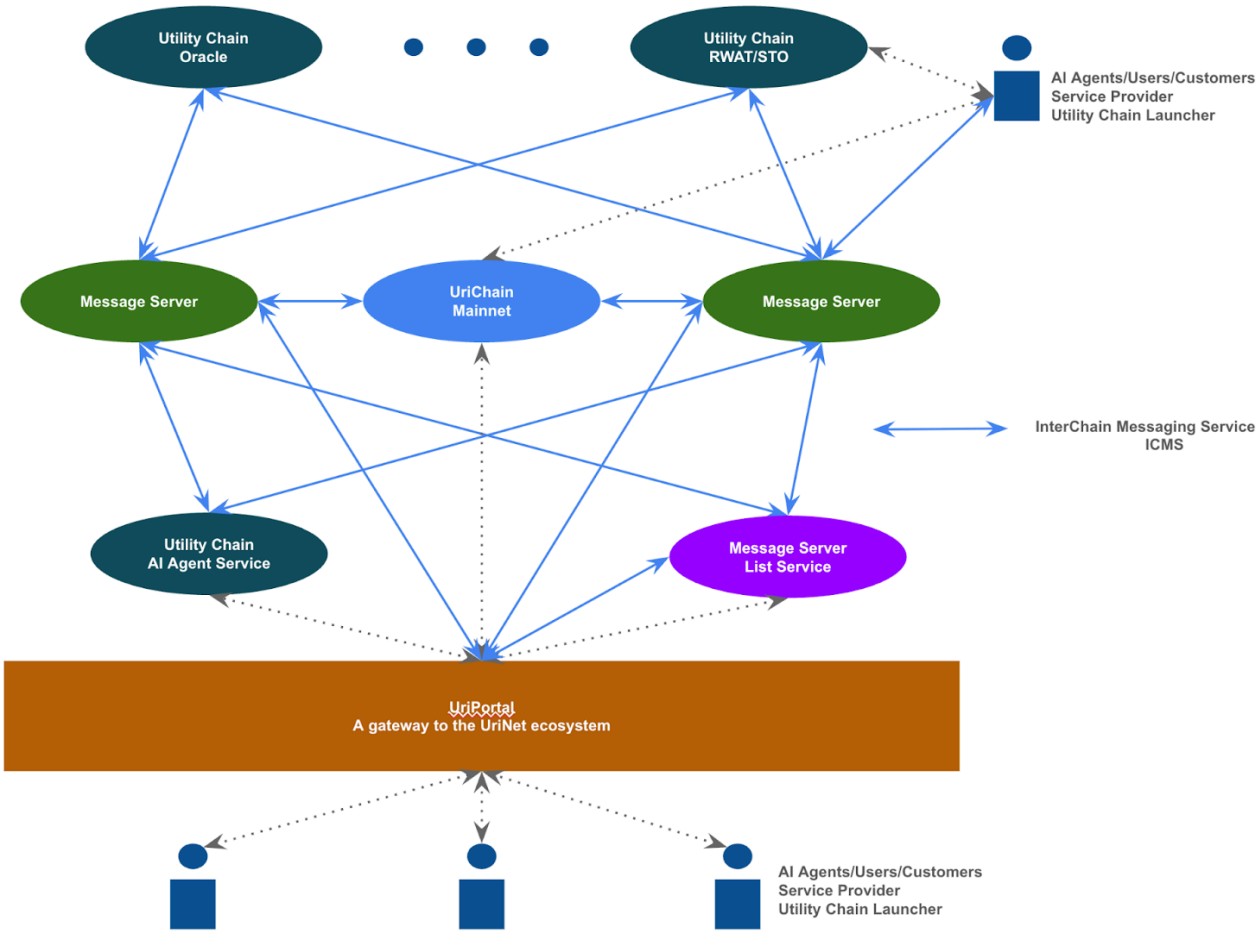
- **UriChain (mainnet core):** The common base layer of UriNet. It anchors the network's shared economics, staking, governance, and core coordination functions, and serves as the root network through which common rails, trust primitives, and shared service standards are standardized and exposed.
- **Utility Chains (product/service specific chains):** Sovereign, single-purpose chains dedicated to a defined product, service, or domain. A Utility Chain may operate as an independent service chain or as part of a broader multi-chain application that composes capabilities from other chains through ICMS. Each Utility Chain controls its own application logic and operational cadence while remaining interoperable with the rest of UriNet. Some Utility Chains may also exist to support agent-oriented functions such as attestation, dispute handling, task routing, negotiation, compliance, or other specialized services.
- **UriPortal (ecosystem gateway):** The shared entry point into UriNet for users, builders, partners, and agents acting through approved interfaces. UriPortal surfaces chain discovery, onboarding, registry data, service metadata, capability discovery, and standardized access to network services so applications can be found, connected, and composed without bespoke infrastructure work for each integration.

- **ICMS (Inter-Chain Message System):** The protocol and service layer for cross-chain coordination in UriNet. ICMS uses standardized interchain messages and supporting message servers so chains can exchange actions and verified results without relying on brittle, one-off bridge logic. In the current design, ICMS supports at least two message patterns: **Type 1 messages** for direct chain-to-chain or module-to-module delivery, and **Type 2 messages** for multi-chain query-and-finalize workflows, where a message server gathers required responses from multiple chains, signs the assembled result, and delivers the final message to the destination chain for execution.
- **URI token (URI):** The native utility token of UriNet and the native coin of UriChain. URI is the standard unit used to access network services, pay fees, participate in staking and governance, and support shared modules across the ecosystem. The broader design direction also assumes URI should remain the common currency of network usage even when applications execute on different Utility Chains, including cases where agent-driven and multichain workflows rely on shared settlement rails.

Design principles

- **Application-first:** UriNet starts from real services and business workflows, then gives them the chain structure they actually need. The network is meant to support both standalone utility chains and applications built by composing multiple specialized chains.
- **Sovereign yet harmonized:** Each Utility Chain remains responsible for its own application logic and local governance, but participates in a common network language for messaging, payments, discovery, and service composition.

- **Composability through ICMS:** Cooperation is not treated as an optional afterthought. UriNet is designed so a chain can call on payment, escrow, oracle, compliance, DEX, negotiation, attestation, or other services across the network through standardized interchain workflows.
- **Off-chain execution, on-chain trust:** UriNet does not assume that every part of an intelligent service should run on-chain. Instead, it focuses on the parts that most benefit from shared trust: permissions, commitments, evidence, settlement, policy enforcement, and final outcomes.
- **Safe automation and bounded delegation:** Agentic workflows should operate within explicit permissions, clear service boundaries, and auditable execution paths. UriNet is designed to support automation while preserving accountability, operator control, and safety rails.
- **Simple first, extensible later:** The current direction favors the simplest safe implementation that can be generalized over time. That applies to utility chain rollout, ICMS operation, and network participation rules.
- **Progressive ecosystem opening:** In the near term, core infrastructure and many initial utility chains are expected to be closely managed by the UriNet organization and community. Broader external participation can expand over time as standards, approval processes, and service-operation rules mature.
- **Web-simple UX and predictable economics:** Users should not need to understand the internal chain topology to use UriNet applications. The network aims to provide discoverable services, clean application flows, and a common URI-based economic layer across the ecosystem.


Utility Chain
Oracle
Utility Chain
RWAT/STO
AI Agents/Users/Customers
Service Provider
Utility Chain Launcher
Message Server
UriChain
Mainnet
Message Server
InterChain Messaging Service
ICMS
Utility Chain
AI Agent Service
Message Server
List Service
UriPortal
A gateway to the UriNet ecosystem
AI Agents/Users/Customers
Service Provider
Utility Chain Launcher

Why AI agents around the world need blockchain

1. A trusted medium for agent-to-agent contract negotiation and finalization is necessary

For completely separate agents to truly collaborate or transact with one another, they need a medium through which they can propose a deal, negotiate terms, and execute it, with rewards or slashing depending on the outcome. Many real-world tasks do not fit into a single fixed template and require negotiation of terms before the transaction can proceed.

For example, the following items may vary from one transaction to another:

- scope of work
- price
- delivery deadline
- timeout conditions
- whether partial payment is allowed
- the type of evidence required
- inspection or verification standards
- the method for selecting an oracle or verifier
- refund rules in case of failure
- bond size
- dispute resolution procedures
- policy and compliance conditions

Accordingly, what the agent economy truly needs is not merely a means of contract execution, but a contract formation layer that includes draft proposal, counterproposal, clause revision, version control, final approval, and finalization of the agreed version.

This layer is necessary so that agents can clearly define the following:

- what exactly has been agreed to
- under what conditions payment is released
- what evidence is considered valid
- who bears which responsibilities
- which version is the final agreed version
- what execution logic that final version is connected to

Why must this be on-chain?

A blockchain provides a single source of incorruptible truth together with programmatic execution.

Disputes between agents do not arise only from the question of whether the work was completed. In practice, disputes arise even more often over what terms were ultimately agreed upon.

Later, one side may claim:

- that document was only a draft
- I never agreed to that clause
- the counterparty changed the contents midway
- the version I approved is different from the version that was actually executed

- the timeout extension or change in inspection standards was never finally accepted

Ordinary server logs can be modified or selectively presented by the operator or by one side. By contrast, if the contract version hash, proposer, counterproposal relationship, approving parties, time of final confirmation, and linked execution state are recorded on-chain, it becomes possible to determine, without trust, which version is the final agreed version.

In addition, a blockchain can directly connect the finally agreed contract state to the following:

- escrow
- milestone release
- verifier-gated payout
- oracle-based release
- dispute path
- slash rule
- timeout refund

In other words, a blockchain does not merely store contract contents. It becomes a uniquely powerful settlement layer that connects contract formation and automatic contract execution into a single verifiable flow.

2. Agent identities and principal-agent relationships must be provable globally

For agents in different organizations or networks to collaborate, there must be a common standard for verifying who the counterparty is, whom they represent, and what qualifications and permissions they have.

Examples:

- Does this agent really represent user A?
- Is this agent an approved purchasing delegate for a certain company?
- Is this agent operating under a specific model or policy profile?

Why must this be on-chain?

An account database on a central server requires trust in the operator. By contrast, DID records anchored on-chain, delegation information, revocation status, and capability information can be verified by multiple organizations without separate trust assumptions. In particular, it becomes difficult to deny later that the authority truly existed at that point in time.

3. Delegation of authority must be scope-limited, revocable, and re-verifiable

AI agents act on behalf of humans or other agents. But such delegation must be limited.

Examples:

- maximum payment limit
- permitted API scope
- expiration time
- whether re-delegation is allowed
- permission only for specific counterparties
- permission only under a specific region or policy

Why must this be on-chain?

All stakeholders must be looking at the same authority state under the same standard. On a central server, the operator can change it and that is the end of it. An on-chain authority state gives everyone the same source of truth.

4. Money and results must be exchanged conditionally and atomically

Agent A performs work, Agent B pays, and in many cases payment should only be released after a human reviewer or oracle confirms the outcome. A perform-first, settle-later model creates disputes.

Examples:

- The result was delivered, but payment was not made
- Payment was made, but the result was poor
- Only partial completion occurred, but full payment was claimed
- The counterparty disappeared in the middle of a long-running task

Why must this be on-chain?

Conditional settlement mechanisms such as escrow, milestone release, timeout refund, and slashable bonds require a neutral state machine. A blockchain can automatically, programmatically, and nearly instantly enforce execution once conditions are satisfied.

5. An oracle is necessary to bring off-chain reality on-chain

If AI agents are to connect to the real world, they must rely on important facts that exist off-chain. In an AI-agent economy, most important facts are outside the chain.

Examples:

- whether delivery was completed
- whether on-site photo inspection passed
- whether a ticket was issued

- news or exchange rates
- sensor state
- results from a specific website
- proof of site visit
- response from an external API

Why must this be on-chain?

These facts move money, authority, and penalties. A single oracle is a single point of failure. If someone injects false data, the whole system is destabilized. Therefore, there must be a structure in which multiple agents or oracle participants submit facts, make attestations, challenge one another, and receive rewards or penalties according to reputation and stake. This process must operate on a publicly verifiable, collectively agreed truth layer, not on truth defined by a central server.

6. When outcomes are wrong, evidence and dispute procedures are necessary

AI agents can produce incorrect outputs, behave maliciously, or violate Service Level Agreements (SLAs). When many agents collaborate in sequence, responsibility becomes ambiguous.

Examples:

- false result submission
- incorrect oracle response
- SLA violation or submission of low-quality output
- incorrect tool call
- double billing

- violation of model commitments

Why must this be on-chain?

It must remain tamper-proof who promised what, what evidence was submitted, and what judgment was rendered. Blockchain records provide an objective reference point for dispute resolution.

7. A global shared state is needed when multiple agents hand off a task or collaborate

For one goal, a planner agent, purchaser agent, verifier agent, human-bridge agent, and settlement agent may operate in sequence.

Examples:

- What stage is the process currently in?
- Who has the next execution right?
- Has it failed?
- Can it be retried?
- Has a timeout occurred?

Why must this be on-chain?

If a single central orchestrator controls the state, then if that operator changes the rules or the server stops, the whole market is affected. Blockchain can maintain a shared workflow state without requiring trust.

8. The truthfulness of computation results must be cryptographically verifiable

An AI agent may claim:

- I computed this with a high-performance model
- This result was produced under a specified policy and model version

But in reality, it may have used a cheaper model or incorrect parameters.

Why must this be on-chain?

The AI inference itself does not need to run on-chain. Rather, the proof and verification of the result should exist on-chain. That is what allows independent verification of whether the computation was actually performed in the promised way. A blockchain can serve as the settlement layer that verifies whether computation was truly performed as promised, in combination with structures such as zk-proofs, TEE attestations, and signed execution receipts.

9. Multi-party approval and shared accountability structures are required

Large asset transfers, sensitive data access, control of physical devices, and execution of dangerous commands should not be left to a single agent. In many cases, multiple agents, or a mixed group of humans and agents, must approve together.

Why must this be on-chain?

Multisig, quorum, weighted approval, and group accounts are only meaningful if everyone sees the same rules and the same state. These multi-party shared-responsibility structures can be operated far more transparently and enforceably on-chain than on a central server.

10. Agent reputation and qualifications must not become the property of one platform

As the agent market grows, the question of who is trustworthy becomes increasingly important. But if reputation data such as what services an agent provides, at what price, with what SLA, with what qualifications, and with what past performance remains confined to one platform's database, it simply recreates a Web2 intermediary structure.

Why must this be on-chain?

Agent registration, reputation, slashing history, completion rate, inspection history, and credentials should live on a shared ledger so that trust becomes public. Blockchain-based reputation is much harder to manipulate because it is tied to activity history and slashing history.

11. Human-centric payment systems do not fit an agent-to-agent economy

AI agents need payment patterns such as:

- per-call micropayments
- usage-based billing
- recurring allowances
- sponsored gas
- prepaid quotas
- subscription streams
- conditional payouts

Why must this be on-chain?

In an AI-agent economy, humans cannot click an approval button every time. Programmable payments and automatic settlement are required. Centralized payment networks are limited in micropayments, censorship resistance, and real-time settlement.

12. Cross-chain and cross-domain failure-handling rules are necessary

If UriNet is composed of multiple utility chains, agent-related state will not remain on a single chain. Identity may be managed on one chain, settlement on another, and reputation on yet another.

If UriNet also connects to external chains and systems such as Bitcoin, Ethereum, and Solana, then success alone is not enough. Failure, timeout, and rollback semantics also matter.

Why must this be on-chain?

UriNet with its ICMS supports arbitrary-bytes transfer, acknowledgements, receipts, timeouts, and cross-chain account control, and it handles these failure semantics at the protocol level. It is difficult to build such trustless failure-handling structures with ordinary server webhooks alone.

13. Emergency halt, recovery, and upgrades must also be jointly controlled

In an AI-agent-driven market, if an incident occurs, a specific function may need to be halted quickly. For example, an oracle source may be contaminated, a particular class of agent may behave maliciously, or a policy may be faulty.

Why must this be on-chain?

Who can stop what, under what conditions, and to what extent is itself part of the trust structure. It must not be a system where a central operator can halt things at will. It must be based on shared rules.

Why AI agents need UriNet

For actually useful and meaningful real-world workflows involving AI agents, there are certain essential functions that must be provided at the blockchain core. Additionally, there are specific services for agents that the blockchain must provide.

A. UriNet provides the following blockchain core

1) Contract Formation and Finalization Layer

A function that finalizes, as a consistent on-chain contract state, the contract drafts, revisions, counterproposals, and final approved versions created through agent-to-agent negotiation.

Example components:

- draft contract registration
- counterproposal submission
- version control
- final approval status
- anchoring of the final-version hash
- linkage to the execution contract

Why must this be on-chain?

The most important thing when parties with different interests transact is that everyone must be able to recognize the same document as the final agreed version. On an ordinary server, the operator or one party can alter negotiation records or selectively present a version. But if contract versions, proposers, approvers, and the time of final confirmation are recorded on-chain, the authenticity and precedence of the final version can be determined without trust.

2) Negotiation State and Version Control Layer

A function that manages the entire contract negotiation process through state transitions and a versioning system.

Examples:

- Draft
- Proposed
- Counter-Proposed
- Accepted
- Rejected
- Expired
- Finalized
- Superseded

For each version, the following should also remain:

- reference to the previous version
- changed clauses
- proposing party

- response status
- approval status
- timestamp

Why must this be on-chain?

Disputes between agents arise not only from the final outcome, but also from who proposed which clause and what the counterparty accepted. This negotiation state and version history must remain tamper-proof so that there is an objective standard for post-facto responsibility determination and dispute resolution.

3) Execution Binding Layer

A function that directly links the final agreed contract state to the actual execution logic.

Examples of linked targets:

- escrow
- milestone release
- verifier-gated payout
- oracle-based release
- dispute path
- slash rule
- timeout refund

Why must this be on-chain?

If the negotiation phase and the execution phase are separated, the final agreed document can differ from the settlement logic that actually gets executed. On-chain, the final approved contract hash can be directly bound to the execution contract state so that the agreed conditions lead directly to automatic enforcement.

4) Agent Identity Anchor

An on-chain identity anchor that distinguishes agent, owner, operator, and service provider. This is not just an address, but includes roles, principal-agent relationships, and policy metadata.

Why must this be on-chain?

Because multiple organizations must be able to verify the same identity without relying on a central onboarding system.

5) Delegation Registry

Records delegation relationships on-chain:

- human → agent
- company → agent
- agent → sub-agent

Included elements:

- scope
- expiry
- spend limit
- whether sub-delegation is allowed

- revoke functionality

Why must this be on-chain?

Authority state must be independently verifiable by anyone, and delegation defines the scope of economic rights and responsibilities.

6) Capability Registry

Manages what an agent can do as on-chain capability objects.

Examples:

- can make payments up to $500
- can access a specific API
- can query a specific dataset
- can control a specific device
- can provide a certain service such as flight booking, restaurant reservation, legal service, or accounting

Why must this be on-chain?

When agents collaborate on-chain, or when a human gives an intent, it must be possible to know which agent is capable of what. For agent services and collaboration, these permissions are scarce rights that can be traded, revoked, restricted, or lent, and they also form a clear list of usable capabilities per agent.

7) Intent Commitment Layer

A layer that commits the user's or a higher-level agent's intent, constraints, budget, and prohibited conditions as structured state.

Why must this be on-chain?

When disputes arise later, a tamper-proof shared standard is needed to determine whether a certain action fell within the original intended scope.

8) Conditional Settlement / Escrow Engine

A settlement layer in which payment is released only after job completion, attestation including oracle input, human review, or quorum approval.

Why must this be on-chain?

Neutral conditional settlement between parties with different interests is best handled by a blockchain state machine.

9) Bond / Stake / Slash Engine

A structure in which service providers, oracles, reviewers, relayers, human workers, solvers, and others post collateral and receive penalties for false reporting or SLA violations.

Why must this be on-chain?

In a global agent network, protocol-level economic sanctions are the strongest way to enforce trust without relying on legal enforcement.

10) Evidence & Provenance Registry

Records evidence objects such as:

- screenshot hashes
- signed API responses
- signed sensor data
- TEE attestations
- human reviewer signatures
- timestamps
- job-log hashes
- hashes of various data sources

Why must this be on-chain?

For dispute resolution and penalty enforcement, the order, integrity, and submission timing of evidence must be preserved.

11) Oracle / Attestation Ingestion Layer

A function that collects facts from the real world from multiple sources and reflects them on-chain according to threshold, weighting, and freshness rules.

Why must this be on-chain?

Once external facts begin to move assets, a shared truth layer is necessary.

12) Workflow State & Receipt Layer

Manages, as shared state, the current stage, outputs, acknowledgements, timeout status, and retry eligibility of multi-step agent workflows.

Why must this be on-chain?

If agents built by different operators are to collaborate without a central orchestrator, they need a shared state.

13) Reputation & Credential Ledger

Accumulates records such as:

- completion rate
- dispute history
- slashing history
- verifier score
- credential status

Why must this be on-chain?

Reputation must become a public trust asset across the whole network, not a monopolized asset of one platform. On that basis, both agents and humans can carry out desired work with credible quality.

14) Sponsored Fees / Paymaster Layer

A function that allows the owner, employer, protocol, or marketplace to pay fees on behalf of the executing user, even when the user does not directly hold gas.

Why must this be on-chain?

In an AI-agent economy, the party that executes and the party that bears the cost may be different. This must be handled safely at the protocol level.

15) Shared Policy & Compliance Layer

Expresses shared rules such as:

- permitted message types
- permitted counterparties
- jurisdiction rules
- human approval requirements
- restricted action policies

Why must this be on-chain?

Because policy is not just an internal setting. It becomes a shared rule that binds transactions, evidence, settlement, and penalties.

16) Multi-Party Approval Layer

A function that requires multi-agent or human-plus-agent approval for high-risk actions such as:

- high-value asset transfers
- slashing
- rule updates

Why must this be on-chain?

Because the results of multi-party approvals must be aggregated and enforced in a tamper-proof way.

17) Proof Verification Layer

A layer that verifies the truthfulness of external computation results using:

- zk proofs
- signed execution receipts
- TEE attestations

Why must this be on-chain?

Heavy AI inference should happen off-chain, but its results require a trustless shared settlement layer for verification.

18) Circuit Breaker & Safe Recovery

A function that provides:

- per-module halts
- halting of specific capabilities
- isolation of specific oracle sources

- resume procedures

Why must this be on-chain?

Automatic and immediate crisis response is important, but crisis-response authority is itself part of the trust structure and therefore requires shared control.

B. UriNet provides the necessary services for agents to execute real-world workflows

1) Agent Contract Negotiation and Finalization Service

A service in which different AI agents propose and revise negotiable contract drafts regarding work, service provision, data access, device control, settlement conditions, and so on, and ultimately finalize a smart-contract state approved by both sides or by multiple parties on-chain.

This service is not merely a contract execution layer. It makes the contract formation process itself traceable and verifiable, and ensures that the final agreed version is directly connected to automatic settlement, evidence submission, review, dispute resolution, and penalty enforcement logic.

Why must this be on-chain?

In an agent market, the real core of disputes lies in the final contract contents and the negotiation history. If versioned contract hashes, revision history, proposers, approvers, and the time of final confirmation remain on-chain, then it is possible to prove without trust what conditions were ultimately agreed upon. In addition, because this final agreed version can be

directly linked to settlement and penalty logic, contract formation and contract execution become unified as one verifiable economic flow.

2) Agent-to-Agent Task Marketplace

A marketplace in which one agent places sub-tasks with another agent.

Examples:

- a research agent hires a translation agent
- a purchasing agent hires a verification agent
- a planner agent calls a human fallback worker

Why must this be on-chain?

Because ordering, acceptance, deposits, result submission, and penalties must operate in a trustless market not monopolized by a specific platform.

3) Human-Bridge Marketplace

A marketplace that connects reality-based tasks unsolved by AI to human workers.

Examples:

- taking on-site photographs
- visit verification
- offline inspection
- item pickup
- simple errands

Why must this be on-chain?

Because human workers are also participants with different interests, so evidence, settlement, reputation, and disputes must run according to public rules.

4) Verification-as-a-Service: Oracle Bounty Service

A service that rewards agents, oracles, or attestors who prove a specific external fact.

Why must this be on-chain?

Because once factual determination has monetary consequences, submission, challenge, rewards, and penalties must all operate transparently and fairly on the ledger.

5) Intent Auction / Solver Market

A market that selects, by auction, the agent that can achieve a given goal most cheaply and safely. The user submits only the goal, and multiple solver agents compete by proposing the best execution path.

Why must this be on-chain?

Because bidding, commitment, winner selection, and payment release must happen neutrally, reducing the possibility of platform manipulation.

6) Data Access Rights Exchange

A service for buying and selling datasets, API access, and knowledge-base access along with conditions such as duration, usage volume, and resale restrictions.

Why must this be on-chain?

Once data access rights are traded, they become an economic right, and ownership, leasing, and revocation require a trustless record.

7) Agent Payment Rail

A payment rail for:

- per-call payments
- per-result payments
- subscriptions
- recurring allowances
- streaming payments

Why must this be on-chain?

Because payments between agents are hard to operate through human-approval-based payment systems, and micropayments, high frequency, automation, and conditional settlement are core requirements. Programmable settlement is required, making centralized payment networks inefficient for this structure.

8) Compliance Attestation Service

A service that attests that a certain task was carried out within the scope of policy, legal requirements, and internal rules.

Why must this be on-chain?

Because regulators, customers, and service providers must share the same compliance evidence, and the history of changes to that evidence also matters.

9) Real-World Device Access Brokerage

A service through which agents can lease access rights to devices needed for real-world tasks from operators who provide drones, cameras, printers, robot arms, vehicles, and IoT sensors.

Why must this be on-chain?

Because access to physical devices is itself an economic right, and when accidents happen, responsibility tracking and compensation structures are also required.

10) Decentralized AI Resource Rental

A market that connects participants with idle compute resources to agents needing computation, while also performing proof of results and automatic settlement. For example, renting out spare AWS EC2 capacity to another company or person. The resources here include AI agents and GPU compute.

Why must this be on-chain?

Connecting global idle resources in a peer-to-peer way without a specific cloud provider requires result submission, settlement, and penalties to interlock without trust.

11) Smart Contract / Tool Audit Bounty

A service in which agents find vulnerabilities in smart contracts or tool chains and receive automatic rewards when conditions are met.

Why must this be on-chain?

Because global participation becomes possible only when rewards are enforced by code rather than by the goodwill of a central company.

12) Shared Procurement Service

A service in which multiple agents perform joint purchasing, joint bidding, or joint procurement.

Why must this be on-chain?

Because cost-sharing, ownership shares, joint approval, and settlement of results must be handled transparently and in a tamper-proof way.

13) Dispute Resolution Service — Dispute Court / Arbitration Service

A dispute resolution service that includes evidence submission, judgment, refunds, slashing, and deposit release.

Why must this be on-chain?

Because settlement and penalties must be automatically enforced after judgment, and the case record itself must not be tampered with.

Why UriNet in General?

Most blockchain ecosystems still force builders into one of two bad choices. Either everything is pushed onto a single general-purpose chain, where unrelated applications compete over the same fees, latency, governance, and execution surface, or applications are spread across separate chains and rollups that do not cooperate cleanly in practice. In both cases, the result is similar: fragmented liquidity, fragmented trust, repeated infrastructure work, and product flows that feel nothing like the web. UriNet is designed for a different objective: real utility applications that can operate independently when they should, and coordinate natively when they must.

The need becomes clearer when the application is not a single closed product, but a service that depends on several specialized functions at once. A tokenization flow may require issuance logic, compliance checks, payment settlement, and escrow. An insurance flow may require oracle inputs, eligibility checks, locked funds, and final policy execution. These are not edge cases. They are normal real-world workflows. UriNet is built around the premise that such applications should not be forced to choose between monolithic design and fragile cross-chain patchwork. Instead, they should be able to use dedicated Utility Chains and coordinate through a standard interchain mechanism.

The problems UriNet addresses

1. **One-chain-fits-all creates unnecessary tradeoffs.**

   General-purpose execution environments force unrelated applications into the same global fee market, governance surface, and performance profile. Real products often need different operating assumptions, different upgrade cadences, and different risk

boundaries. UriNet addresses this by giving each service its own Utility Chain while keeping that chain connected to shared network rails.

2. **Cross-chain cooperation usually remains optional and brittle.**

   Many ecosystems support interoperability only through ad hoc bridges, custom integrations, or one-off service agreements. That is not sufficient for applications that depend on multiple chains for normal execution. UriNet addresses this through ICMS, which treats interchain coordination as a first-class network function rather than an external add-on.

3. **Multichain applications need orchestration, not just transport.**

   Moving a token from one chain to another is not enough when the destination application needs verified results from several chains before it can execute safely. ICMS is designed to support both direct chain-to-chain messages and multi-query workflows in which message servers gather the required responses, assemble the final message, sign it, and deliver it to the destination chain. This avoids blocking application execution on synchronous cross-chain reads and reduces the fragmentation that would otherwise appear in multi-step services.

4. **Builders spend too much effort recreating common infrastructure.**

   Payments, discovery, service access, oracle connectivity, registry metadata, and cross-chain coordination should not have to be rebuilt for each application. UriNet centralizes these common rails at the network level through UriChain, UriPortal, and associated ecosystem services so builders can focus on product logic instead of stitching together basic infrastructure.

5. **Real-world services need shared trust and shared economic rails.**

   Financial and utility applications need predictable fees, recognizable settlement logic, controlled asset movement, and clear trust assumptions across service boundaries.

UriNet keeps URI as the common utility asset for network usage, staking, and shared services, while designing cross-chain asset movement and application usage around standardized routes, acknowledgements, and safety controls.

Our approach

- **Application-first architecture:**

  Each product or service can run on its own Utility Chain with its own logic and operating cadence, instead of being forced into a single global execution environment.

- **Shared rails at the network layer:**

  UriChain and UriPortal provide the common economic, coordination, discovery, and service-access foundation for the ecosystem, so each application does not need to rebuild core infrastructure from scratch.

- **ICMS for native coordination:**

  UriNet's interchain model is meant to support both direct interchain actions and multi-chain dependent workflows. Type 1 messages handle direct delivery. Type 2 messages support query-and-finalize patterns for applications that need inputs from several chains before final execution.

- **Sovereignty with managed interoperability:**

  Utility Chains remain distinct execution environments, but they participate in a common message, registry, and service framework. This allows local control without losing ecosystem coherence.

- **Predictable URI-based economics:**

  URI remains the common utility unit for fees, staking, governance participation, and shared-service usage across the network. This gives builders and operators a clearer

economic model than a system where every interaction depends on unrelated external assets or fragmented fee standards.

- **Simple first, broader later:**
  UriNet is being designed around the simplest safe implementation that can be generalized over time. That includes staged utility chain rollout, staged operation of interchain message servers, and a gradual expansion from initially managed infrastructure toward broader participation under defined standards.

Who it is for

UriNet is intended for applications that benefit from dedicated execution but cannot remain isolated from the rest of the network. That includes tokenization and security-token platforms, escrow and settlement services, oracle-dependent applications, benefits and loyalty systems, advertising and payout systems, media and provenance services, and insurance or risk products that require coordinated state across multiple service chains.

What this enables

- **Cleaner product architecture:** teams can separate services into purpose-built chains without giving up coordinated execution across the ecosystem.
- **Reusable network effects:** each additional Utility Chain can provide capabilities that other applications can call through common interchain patterns.
- **Lower integration burden:** shared discovery, service metadata, access points, and message-server infrastructure reduce bespoke integration work.

- **A more credible operating environment for partners:** applications can rely on standardized rails, staged governance, and clearer trust boundaries instead of one-off bridge arrangements and opaque cross-system dependencies.

UriNet exists because real applications need more than blockspace. They need dedicated execution, shared rails, predictable economics, and a native way to coordinate across specialized services. That is the gap UriNet is designed to fill.

Design Principles

UriNet is engineered for real-life utility. UriNet is onboarding real world companies addressing and solving real world use cases. Through collaboration with these strategic partners, we are able to work together and fully utilize UriNet to address the real needs of real customers in these various fields. Blockchain is no longer “the solution looking for a problem.”

The principles below guide architectural choices, product design, and ecosystem rollout. They define what the network is optimizing for; the following UriNet Architecture section explains how those principles are implemented in practice.

1. Real products come first.

Each service should have the chain structure it actually needs, whether that is a self-contained Utility Chain or a multichain-dependent application that coordinates with other specialized chains. The goal is to give each product clear execution boundaries, a clear upgrade path, and a clean place for its own logic, instead of forcing unrelated workloads into one shared environment.

2. Modularity at both the chain and service layer

UriNet is modular in two senses. First, a Utility Chain should include only the modules required for its own purpose. Second, application capabilities can be composed across chains through standardized network services rather than duplicated everywhere. This lets builders keep product logic local while still using shared capabilities such as settlement, escrow, oracle data, compliance attestations, and other ecosystem services when needed.

3. Sovereign yet harmonized

Each Utility Chain remains a distinct execution environment with control over its own logic, cadence, and local governance. At the same time, chains that participate in UriNet should speak the same interchain language, register into the same ecosystem framework, and operate against common standards for messaging, service access, and network participation. Local control should not come at the cost of ecosystem coherence.

4. Interchain coordination is a primitive, not an add-on

UriNet is designed on the assumption that many useful applications depend on other chains during normal execution. For that reason, interchain behavior is not treated as a bridge-like afterthought. Through ICMS, chains can exchange standardized interchain messages for both direct delivery and coordinated multi-chain workflows. This makes cross-chain action part of the application model itself.

5. Shared rails should exist once and be reusable everywhere

Common rails such as coordination, discovery, service metadata, and access pathways should be provided at the network level instead of rebuilt by every application. UriChain provides the common economic and governance base, while UriPortal and related registry and access services make chains and services discoverable and callable across the network. As the

ecosystem grows, shared infrastructure should reduce repeated integration work rather than multiply it.

6. URI should remain the common utility asset of the network

UriNet is designed around URI as the standard utility asset for fees, service usage, staking, and governance participation. Even when applications run on separate Utility Chains, the economic model should continue to reinforce URI as the common network currency. Cross-chain asset movement therefore has to be designed with safety, accounting integrity, and operational simplicity in mind, rather than allowing fragmented payment standards to emerge by default.

7. Scale by specialization, not by forcing everything into one chain

UriNet scales horizontally by adding Utility Chains and by separating services into purpose-built execution environments. Capacity should expand where demand exists, and applications with different workloads should not have to compete in a single global mempool or fee market. This principle applies not only to throughput, but also to governance scope, upgrade cadence, and operational risk.

8. Performance should feel simple to the user and operable to the network

The network should aim for responsive application flows, predictable costs, and clear end-to-end execution paths. But performance is not only about block times. It also depends on whether cross-chain workflows can be reasoned about, routed reliably, and completed without unnecessary fragmentation. UriNet therefore treats operational clarity, bounded message flows, and manageable coordination paths as part of performance design, not separate concerns.

9. Simplicity over cleverness

UriNet should prefer the simplest safe design that works and can later be generalized. Clear interfaces, auditable message flows, bounded operational assumptions, and understandable

token mechanics are more valuable than theoretically elegant but fragile designs. Real systems need to be deployable, governable, and maintainable before they can become broadly open and composable.

Taken together, these principles define UriNet as an application-first network of specialized chains coordinated through shared rails and standardized interchain execution. The result is a system that aims to stay modular, scalable, and operationally coherent while allowing new chains and services to strengthen the broader ecosystem rather than fragment it.

UriNet Architecture

UriNet is organized as a coordinated network of three architectural layers: **UriChain (mainnet)**, a set of **Utility Chains**, and shared **ecosystem infrastructure through UriPortal and ICMS**. Together, these layers provide execution, coordination, settlement, service access, and cross-chain interoperability as native parts of the network rather than as after-the-fact integrations.

UriChain (Mainnet)

UriChain is the shared core of UriNet. It serves as the network's common economic and governance base, anchors shared coordination and settlement functions, and provides the primary home of the **URI** token. It is also the layer through which network-wide services and shared incentives remain harmonized even as applications run across separate Utility Chains.

Utility Chains

Utility Chains are sovereign, single-purpose blockchains that execute product logic or provide specialized network services. A Utility Chain may be a standalone chain, or it may be one component of a broader multichain-dependent service that interacts with other chains through UriNet's interchain coordination model. The architecture favors clear scope per chain so that execution boundaries, governance responsibilities, and operational roles remain understandable.

Two broad categories of Utility Chains are expected.

1. Community/Foundation Essential Chains

These are foundational service chains that provide reusable network capabilities for the broader ecosystem. Early examples include service chains such as:

- A **Real World Asset Tokenization (RWAT)/Securities Tokenization (ST)** chain for the tokenization of real world assets
- An **Escrow** chain for conditional holding and release logic. Escrow provides conditional holding, release, refund, and cancellation logic for value transfers and service settlement. It is intended for cases where payment must not finalize immediately, such as loans, milestone delivery, bilateral trade, insurance premium handling, claim payouts, staged disbursements, and any workflow in which payment depends on later approvals, time conditions, or verified external facts.
- A **DEX** chain for specialized exchange and liquidity functions. The DEX service provides exchange between URI, which remains the native coin of UriChain, and service-specific tokens issued by Utility Chains.
- An **Oracle** chain for external data and attestations. The Oracle service delivers external data and attestations to other chains through ICMS. It operates as a specialized Utility

Chain that receives requests, aggregates signed oracle responses, and returns a final on-chain report with acknowledgement. When oracle data must be combined with state from multiple chains, that broader coordination should use ICMS Type 2.

- An **ADs network** chain to handle advertisements, incentivization and rewards, campaign economics, and ad revenue sharing.
- A **Stable Coin Chain** to serve as a standardized method of currency.

2. Utility Chains for Supporting Agentic Workflow

These are foundational service chains that are specifically tailored to natively support agentic workflow. Early examples include service chains such as:

- **Oracle / Real-World Attestation Utility Chain:** Hosts the marketplace and dispute process for real-world fact attestation, including submissions, challenges, staking, scoring, freshness markets, and source weighting. This domain has its own participant set, rewards, penalties, and update cadence, so isolating it improves security and allows more specialized governance.
- **Dispute Resolution / Arbitration Utility Chain:** Handles case creation, evidence scheduling, juror or arbitrator selection, appeals, procedural rules, judgment records, and ruling finalization. Dispute resolution is a distinct institutional subsystem with its own governance model, incentive design, and evidentiary requirements.
- **Agent Task Marketplace Utility Chain:** Runs the open market where agents post tasks, accept work, bid, subcontract, submit results, and settle through the common settlement rails. As a high-volume economic environment with its own fee logic, anti-spam design, and market structure, it benefits from operating as a dedicated chain.

- **Human-Bridge Work Utility Chain:** Supports human fallback tasks such as physical verification, pickup, local inspection, delivery confirmation, and on-site evidence collection. Human labor markets have distinct fraud patterns, service standards, identity requirements, reputation models, and jurisdictional constraints, which makes separate operational boundaries appropriate.
- **Solver / Intent Auction Utility Chain:** Hosts solver competition for user intents, route proposals, cost estimates, safety commitments, and winner selection. Intent solving behaves as a specialized auction market with high-frequency bidding logic and rapidly evolving solver strategies.
- **Data Access Rights Exchange Utility Chain:** Supports the buying, leasing, lending, revoking, and enforcement of access rights for datasets, APIs, proprietary knowledge bases, and usage quotas. Data-rights markets involve distinct pricing models, license formats, revocation rules, and compliance requirements, making them better suited to a specialized chain.
- **Device Access Brokerage Utility Chain:** Brokers rights to use drones, cameras, robot arms, printers, vehicles, sensors, and industrial equipment. Physical device markets involve hardware-specific safety rules, maintenance windows, reservation scheduling, incident handling, insurance logic, and real-world legal exposure.
- **Agent Resource Rental Utility Chain:** Hosts markets for agent-runtime rental, inference service commitments, compute proofs, and performance SLAs.
- **Compliance Attestation Utility Chain:** Operates attestation workflows for regulatory and policy compliance, including sector-specific rule sets, auditor participation, jurisdiction packs, and versioned compliance outputs. Compliance changes frequently across sectors and regions, so a separate chain can govern its own schemas, authorities, upgrade tempo, and audit processes.

- **Reputation / Qualification Utility Chain:** Builds richer agent profiles, domain-specific scores, reviewer networks, qualification badges, and performance markets on top of the common credential core. The canonical trust record can remain in a shared module, while more contested and fast-evolving reputation logic is handled on a separate chain.
- **Shared Procurement Utility Chain:** Supports group purchasing, joint bidding, pooled budgets, ownership splits, and post-award coordination among multiple agents or organizations. This is a specialized coordination market with its own workflow rules, cost-sharing logic, and joint-governance patterns.
- **Smart Contract / Tool Audit Bounty Utility Chain:** Lets agents and humans submit vulnerability reports, claim bounties, post review opinions, and manage disclosure procedures. Bounty markets have their own disclosure rules, severity standards, reviewer incentives, and payout models, making them better suited to a specialized service chain.
- **Payment Rail / Streaming Settlement Utility Chain:** Optimized for high-frequency micropayments, per-call charging, subscriptions, streaming allowances, agent wallet management, and netting. The base chain should still provide escrow and settlement primitives, but a dedicated payment chain may be better suited for throughput, fee tuning, sponsored execution, and agent-to-agent billing at scale.
- **Smart Contract Formation / Negotiation Utility Chain:** Hosts the live contract-formation layer between agents, users, and service providers, including proposal exchange, counterproposals, clause edits, draft versioning, approval routing, and final agreement packaging before anchoring the result into the common contract formation module.

  The shared module should still exist as the canonical trust layer for finalized terms, approval proofs, version references, and execution binding. The negotiation process itself is a higher-churn service domain with its own privacy needs, template libraries,

policy packs, workflow rules, and sector-specific standards, so a dedicated Utility Chain can support richer pre-finalization negotiation without overloading the common core.

3. General-Purpose, Product-Specific Chains

These are purpose-built Utility Chains designed to serve real-world products and service domains. Rather than existing as generic branches of the network, they are intended to support specific application needs such as financial products, payments, benefits, media, advertising, insurance, and other focused use cases where clear execution separation is valuable.

Common characteristics of Utility Chains

- **Sovereign execution with shared rails**

  Each Utility Chain remains a distinct execution environment with control over its own logic, application state, and local governance, while still participating in the shared economic and coordination framework of UriNet.
- **Single-purpose or tightly scoped design**

  Utility Chains are intended to keep a clear functional scope, whether they are product chains or reusable service chains. This improves maintainability, operational clarity, and cross-chain composability.
- **ICMS-native interoperability**

  Utility Chains are designed to interoperate through ICMS rather than through ad hoc bridge patterns. This allows services to coordinate explicitly across chains using standardized message flows, acknowledgements, and follow-up execution paths.
- **URI-anchored network economics**

  Utility Chains may support local application assets where appropriate, but URI remains

the common utility asset for network-level fees, staking, governance participation, and shared-service usage across UriNet.

Who can build utility chains?

In our whitepaper version 0.1, we envisioned an ecosystem where anyone could “plug and play” various UriNet modules, register their chain (after staking) through UriPortal, and be embraced by the greater UriNet ecosystem.

However, after a lot of difficult consideration, we have decided to make some changes.

Our core mission and guiding principle has always been real world usage—that means, not just real world utility, but actually easily being able to use that real world utility in everyday life. Thus, a necessary condition for this is high quality tools and infrastructure, serving and addressing **real world needs** with **real customers**.

Thus, in the initial launch stages of the UriNet ecosystem, utility chain development will be focussed on collaborating with leading strategic partners in their fields who currently have real customers and real needs that UriNet is poised to solve.

As UriNet matures alongside the services that live on it, we plan to open up Utility Chain development to be more permissionless, as protection and quality standards evolve and become more clearly defined—and enforced by the greater community and from the protocol.

ICMS (Inter-Chain Messaging System)

ICMS is the network's native interchain coordination system. It is not treated as a peripheral bridge or a narrow token-transfer rail, but as a core mechanism for how chains within UriNet communicate, coordinate actions, and complete multichain workflows.

In the current design direction, ICMS supports two broad patterns:

- **Direct interchain messaging**, where one chain sends a message or request to another chain for delivery and handling
- **Coordinated multichain execution**, where a workflow depends on information or responses from multiple chains before a final message is assembled and delivered onward

This second pattern is where the **message server layer** becomes important. Rather than requiring synchronous cross-chain reads, a message server can gather the required responses, assemble the final result, sign or attest to the assembled message as defined by the network's operating rules, and deliver that finalized message to the destination chain. This keeps multichain workflows asynchronous, explicit, and easier to reason about operationally.

ICMS is therefore central not only to cross-chain messaging, but also to service composition across escrow, oracle, exchange, token, and application chains.

Message Server

The message server is a fundamental part of the UriNet ecosystem. It acts as a trusted and guaranteed service that delivers interchain messages. For more complex multi-chain workflows, it handles all of the interchain querying and data collection, packaging a singular final result for the final destination chain.

Message Server List Service

MSLS is the discovery layer used by ICMS to find reachable message servers for a target chain, service, or query class.

Interchain Access Point Service

The InterChain Access Point Service stores the actual utility chain access information needed to listen to events or send transactions, such as IP, node name, supported services, protocol version, and fees.

UriPortal

UriPortal is the ecosystem-facing gateway layer of UriNet. It provides the common interface through which participants can discover network services, access shared APIs and developer pathways, and understand what capabilities exist across the ecosystem. In practical terms, UriPortal is the layer that makes the multichain network usable as one coherent system rather than as a disconnected set of chains.

Its role includes functions such as:

- service discovery and ecosystem navigation
- standardized access points and developer-facing interfaces
- service metadata, documentation, and integration pathways
- analytics and ecosystem-level visibility into network usage and service health
- a common surface through which shared rails are made usable across chains

URI Token

URI is UriNet's native utility token and the native coin of UriChain. It is the common utility asset of the network and the primary unit used for fees, staking, governance participation, and access to shared services. URI-denominated economics are intended to keep network-level costs predictable and to preserve a common economic language across otherwise separate Utility Chains.

Consensus

UriChain uses a modified Byzantine-fault-tolerant delegated proof-of-stake structure centered on URI-staked validators. Utility Chains can use consensus and security arrangements appropriate to their purpose, including more independent operation or models tied more closely to UriChain-based assurance. This allows the network to support different service profiles without forcing one uniform security model onto every chain.

Technology: How UriNet Works Under the Hood

DISCLAIMER: As stated in our primary disclaimer, the technical details in this section are preliminary, may be incomplete or inaccurate, and are subject to change without notice. This section in particular is under active development and may be wrong; do not rely on it as advice, an offer, or a commitment. In any case of inconsistency, the primary disclaimer governs and supersedes this section.

This section describes UriNet's internal mechanics at a basic level.

Nomenclature and Roles

Networks

- **UriNet:** The full ecosystem composed of UriChain, Utility Chains, ICMS, and UriPortal.
- **UriChain (mainnet):** The shared core of the network. It anchors URI, validator staking, governance, network-level registration, and common settlement and accountability rules.
- **Utility Chains:** Sovereign, single-purpose chains that execute product logic or provide specialized network services. They interoperate through ICMS.
- **UriPortal:** The ecosystem interface layer for onboarding, discovery, registry access, analytics, and developer-facing infrastructure.
- **ICMS (Inter-Chain Messaging System):** The native protocol used for cross-chain requests, responses, acknowledgements, and related coordination across UriNet.

- **Message Servers / MSLS:** Supporting infrastructure for ICMS routing, discovery, and cached knowledge of available endpoints. These improve communication efficiency but do not replace chain-level verification.
- **Interchain access point service:** This service stores the actual utility chain access information needed to listen to events or send transactions, such as IP, node name, supported services, protocol version, and fees.
- **Message server discovery algorithm:** The service that allows message servers to find routes to other chains across UriNet that they themselves do not directly serve.

Actors

- **UriChain Validators:** URI-staked validators that secure UriChain and participate in core network consensus and governance.
- **Delegators:** URI holders who delegate stake to validators.
- **Utility Chain Validators / Operators:** Parties that run and secure individual Utility Chains.
- **Message Server Operators:** Operators that participate in ICMS routing and response handling.
- **Service Providers:** Maintainers of Utility Chains or network-facing services.
- **Reporters / Oracle Operators:** Parties that fetch off-chain data and submit attestations where needed.
- **Users / Applications:** End users, dApps, and contracts that submit transactions and invoke services across the network.

Identifiers and Types

- Canonical objects include ChainID, ServiceID, Address, Denom, Amount, Height, Time, MessageID, URI, Hash, Signature, PubKey, and Timeout fields.

UriChain Core

UriChain is the shared core of UriNet. It provides the network's common economic base, validator-backed trust anchor, governance layer, and canonical registration and policy layer. Application execution and specialized services generally belong on Utility Chains, while UriChain provides the common foundation that lets them interoperate coherently.

Consensus and Validator Set

UriChain uses a Modified BFT-DPoS model.

- **Validators:** stake URI and produce blocks.
- **Delegators:** delegate URI to validators and contribute to network security.
- **Safety:** holds while Byzantine behavior remains below one-third of voting power.
- **Liveness:** holds when at least two-thirds of voting power continues to participate.
- **Slashable behavior:** includes equivocation, persistent unavailability, and provable network-level or cross-chain misbehavior where UriChain is the enforcement anchor.
- **Operational role:** UriChain prioritizes reliability, coordination, and network accountability over hosting all application execution directly.

Governance

UriDAO governs network-wide parameters such as validator policy, fee policy, registry rules, and shared interoperability standards. The **UriNet Foundation** supports the network's long-term stewardship, ecosystem development, and administrative continuity, while Utility Chains remain sovereign over their own local execution and service-specific rules.

Inter-Chain Messaging System (ICMS)

ICMS is the native interchain communication layer of UriNet. It carries authenticated requests, responses, acknowledgements, and related metadata among UriChain and Utility Chains. ICMS is designed to let chains interact through one common messaging framework rather than through isolated pairwise integrations.

ICMS separates transport from execution. Message servers and discovery infrastructure help locate and forward messages, but they do not decide state transitions on behalf of chains. The destination chain remains responsible for validating the message, checking timeouts and duplicates, applying its own authorization rules, and deciding whether to execute or reject the requested action.

At a high level, ICMS supports two message classes:

- **Type 1 - Simple interchain messages:** direct delivery from one chain to another for a single action, request, notification, or acknowledgement.

- **Type 2 - Multi-query messages:** a higher-order workflow in which one chain initiates a message that depends on results from multiple other chains before a final message can be delivered to the destination chain.

ICMS roles

ICMS introduces a small set of recurring roles:

- **Source chain:** the chain that originates the interchain request

- **Destination chain:** the chain that receives the final executable message
- **Queried chain:** any intermediate chain asked to return data or a decision
- **Message server:** an authorized operator that delivers cross-chain messages, gathers Type 2 query results, assembles the final bundle, signs it, and forwards it to the destination chain
- **MSLS (Message Server List Service):** the service layer that publishes or resolves the eligible message servers for a given workflow or route

The destination chain remains the final execution authority for its own state. A message server does not directly finalize destination-chain state; it assembles and attests the result bundle that the destination chain must still verify before executing.

Type 1: Simple Interchain Message

Type 1 messages are the base transport primitive of ICMS. A Type 1 message carries one committed instruction or notice from one module on a source chain to one module on a destination chain, without requiring the source chain to synchronously wait on the destination chain during execution. Type 1 is defined as a simple module-to-module delivery: a message server detects an ICM-related event emission on the source chain, resolves a route using its cache or the Message Server List Service, forwards the message subject to path and hop restrictions, and the destination module validates the delivering message server's signature and metadata before processing or emitting an error event.

Big-picture flow of a Type 1 ICMS message

1. **ICMS request emission.** The source chain completes its local transaction and emits a structured Type 1 ICMS request event declaring the source module, destination chain,

destination module, payload type, timeout bounds, and routing constraints.

2. **Message-server observation.** A message server connected to the source chain listens for committed ICMS-related events from trusted chain endpoints, detects the new request, decodes it, and converts it into a transport work item.

3. **Route discovery.** The message server determines how to reach the destination by first consulting its local cache table and, if necessary, querying the Message Server List Service and the relevant chain access-point information.

4. **Relay Enveloping.** The message server packages the request into a relay envelope that preserves the original message, attaches routing metadata such as delivery path and hop count, and signs the message for authenticated transport.

5. **Relaying.** If the destination is not directly reachable, the message is forwarded to another message server that can handle the next leg. Each relay checks the route, updates path metadata, and enforces hop-limit rules to prevent endless ping-pong.

6. **Submission to the destination chain.** The final delivering message server submits a normal inbound transaction to the destination chain, carrying the Type 1 message and the delivering server's signature.

7. **Destination-side transport validation.** The destination chain verifies that the delivering message server is legitimate, checks the signature and metadata, confirms timeout and

destination-chain correctness, and performs replay protection.

8. **Routing to the destination module.** Once transport-level checks pass, the destination chain routes the message to the declared target module.

9. **Application-level execution.** The target module validates the message contents according to its own business rules and either executes the requested action or rejects it.

10. **Acknowledgement or error handling.** If execution succeeds, the destination emits an acknowledgement or response event if required. If validation or execution fails, it emits an ICMS error event. That outcome can then be returned to the source side through the same message-server transport model.

Visual summary: Type 1 ICMS message flow

```
Source Chain        Message Server 1          Message Server 2        Destination Chain

-----------------------------------------------------------------------------------------------------------------------------------

|<---- Event Subscription----|


|----ICMS REQ Emission-->|


                                  | MESSAGE SERVER OBSERVATION


                                  |<----ROUTE DISCOVERY----->|


                                  | RELAY ENVELOPING


                                  |------- Forwarding ------------->|
                                                                      |------- Delivery ----->|
                                                                                              |
                                                                                     Execution of Logic
                                                                                              |


          Acknowledgement and response travel back in reverse with validation

|<------------------------------------------- \\ ------------------------------------------------|
```

Type 2: Multi-Query Message

Type 2 messages are used when a final transaction on one Utility Chain depends on information or prerequisite action results from multiple other Utility Chains. Instead of making the destination chain halt, wait, or fragment the business flow across multiple partially dependent transactions, the requester submits one structured Type 2 request to a message server. The message server then queries the required Utility Chains, gathers and verifies the responses, assembles a single final transaction package, signs it, and delivers it to the final destination chain for one complete execution step.

For example, assume a requester wants to execute settlement.execute_purchase on rwa-settlement-1. That final transaction requires three external results before it can execute safely:

- a price quote from oracle-chain-1
- an investor eligibility result from compliance-chain-1
- an asset-availability result from custody-chain-1

Under Type 2, those cross-chain dependencies are collected outside the final destination chain by the message server, so that rwa-settlement-1 receives one signed, complete, self-contained input package and can execute without waiting on remote chains during its own transaction processing.

Big-picture flow of a Type 2 ICMS message

1. **Requester submission.** A user or frontend service provider sends a structured Type 2 request to a message server, declaring the final destination chain, the target Utility

Chains, the target modules, the per-chain query or action payloads, timeout bounds, and fee or tip fields.

2. **Admission and validation.** The message server validates metadata, protocol compatibility, timeout bounds, destination information, and other basic admission rules. Invalid requests are rejected early.
3. **Working-table creation.** If admitted, the message server creates an internal working record, generates a unique aggregate ID, and materializes the list of required query legs.
4. **Route and cache preparation.** For each leg, the message server determines how to reach the target chain, consulting its cache, the Message Server List Service, and chain access-point information as needed. It may also satisfy some legs from a valid response cache when policy allows.
5. **Subrequest dispatch.** The message server signs and sends the required per-chain subrequests to each target Utility Chain, either directly or through another message server.
6. **Response observation and verification.** The message server observes the corresponding committed response events, validates the metadata and timeouts, and stores each result against the aggregate request.
7. **Aggregation and final-payload construction.** Once all required legs are complete, the message server combines the responses into one final payload for the destination chain and signs the final delivery transaction.
8. **Submission to the final destination chain.** The message server delivers the final combined transaction to the destination Utility Chain.
9. **Destination-side validation and execution.** The destination chain verifies the message-server credentials, signature, metadata, completeness, and replay conditions, then routes the request to the destination module for one local execution.

10. **Completion, acknowledgement, and timeout handling.** Success, failure, and timeout are recorded cleanly. If the aggregate request times out before all required legs are complete, the message server marks it accordingly and can return a timeout result to the requester.

Visual summary: Type 2 ICMS message flow

```
Requester / Frontend     msg-server-01      oracle-chain-1   compliance-chain-1   custody-chain-1   rwa-settlement-1
-------------------
```

Message Server

Message servers are the operational layer that turns ICMS from a protocol definition into a functioning interchain system. They sit between chains as authenticated transport and coordination services: listening for committed ICMS-related events, discovering routes to destination chains or peer message servers, signing and delivering outbound messages, and carrying acknowledgements or error results back through the network.

In a simple Type 1 flow, a message server detects a committed interchain message request, determines how to reach the destination, forwards the message directly or through another message server when necessary, and signs the final delivery so the receiving chain can verify that the message came through a legitimate interchain transport path.

In a Type 2 flow, the message server takes on an additional coordination role: it receives a multi-query request, issues and tracks the required subrequests across multiple Utility Chains, gathers and validates the responses, and then creates and signs a single final transaction package for the destination chain. This is what allows the destination chain to execute with a complete dependency set already attached, rather than halting or fragmenting execution while waiting on remote chains.

Beyond transport, message servers are also part of UriNet's operational fabric. They maintain route and response caches, consult the Message Server List Service and chain access-point information when they do not already know a destination, and perform basic health checks against the chains and peer servers they depend on. Their signatures are central to trust in ICMS delivery, so participating chains must know which message servers are authorized to serve them, whether through direct configuration, recognized operator credentials, or startup

discovery of the servers assigned to that chain. Over time, message-server operation can evolve from an organization-managed service into a broader service layer with qualified operators, but their purpose remains the same: to provide the reliable interchain transport, discovery, and aggregation layer that allows UriNet's Utility Chains to function as one coherent network rather than as isolated blockchains.

Message Server List Service (MSLS)

The Message Server List Service is the discovery layer for ICMS transport. Its purpose is to let message servers, chains, and external requesters determine which message server can handle delivery for a given Utility Chain, what protocols that server supports, what service classes it offers, and how it can be reached. In the ICMS sections above, Type 1 and Type 2 delivery both depended on route resolution through a local cache first and the Message Server List Service second. This section makes that discovery layer explicit and defines it in a way that is consistent with those earlier flows.

MSLS is intentionally narrower than the general chain registry or broader ecosystem discovery surfaces. The chain registry records the canonical existence and metadata of chains. The InterChain Access Point Service provides the concrete access information needed to listen to chain events or submit transactions. MSLS sits between those layers: it indexes message servers themselves, the chains they serve, the protocol versions they support, and the service roles they provide, so that ICMS transport can find the correct next hop quickly and deterministically.

Big-picture role of MSLS

1. **Message-server registration.** Each message server registers its identity, alias, endpoint, supported protocol versions, service roles, fees, and served-chain bindings.
2. **Chain-to-server indexing.** MSLS maintains the mapping from a destination chain to the message servers that can handle delivery for that chain.
3. **Server-to-chain indexing.** MSLS also maintains the reverse mapping from each message server to the chains it serves.
4. **Startup discovery.** When a chain or a message server starts, it can query MSLS to discover the relevant message servers for the chains it participates in.
5. **Route lookup during ICMS delivery.** When a message server needs to reach a chain it does not directly handle, it checks its local route cache and then queries MSLS for a candidate next-hop server.
6. **Protocol and service matching.** Route selection is filtered by protocol compatibility, supported ICMS roles, and optional fee hints.
7. **Cache-assisted performance.** Message servers locally cache MSLS responses with expiration to avoid repeated public lookups on every message.
8. **Type 1 support.** For simple messages, MSLS helps select the next hop when the destination chain is not directly reachable from the current server.
9. **Type 2 support.** For multi-query messages, MSLS helps resolve the path for each dependency leg and for the final aggregate delivery.
10. **Discovery (not trust finality).** MSLS helps nodes find candidate message servers, but destination chains still authenticate the delivering server through public-key and credential checks before accepting delivery.

Universal Cross-Chain URI Transfer

DISCLAIMER: As stated in our primary disclaimer, the technical details in this section are preliminary, may be incomplete or inaccurate, and are subject to change without notice. This section in particular is under active development and may be wrong; do not rely on it as advice, an offer, or a commitment. In any case of inconsistency, the primary disclaimer governs and supersedes this section.

Universal Cross-Chain Payments must do more than move a message across chains. For native URI, the transport layer and the asset-accounting layer have to work together so that value can move across UriNet without breaking supply integrity, without creating ambiguous outcomes, and without forcing the destination chain to halt while it waits for remote confirmation. The transport path uses ICMS, but the monetary truth of the transfer lives in a dedicated CoinTransfer module whose job is to reserve, mint, lock, unlock, revoke, account, and close each transfer deterministically. URI remains the network's native utility token and standard unit across the network, while Utility Chains managed by the UriNet organization use local URI-denominated representations so cross-chain payments remain predictable and composable.

For native-coin movement, the design must be conservative. Minting on a Utility Chain is powerful and therefore dangerous if it is not rigidly bounded by backing, acknowledgements, replay protection, timeout handling, and emergency controls. The system therefore uses a fixed 1:1 conversion rule between URI on UriChain and the destination-chain representation URI-<UtilityChainName>, maintains explicit tracking of minted, burned, locked, and circulating

balances, supports emergency stop controls, and can impose stronger release policies such as validator multisignature approval, timelock, or vesting for large or sensitive transfers. Because the CoinTransfer module is a shared protocol surface that must behave consistently across UriChain and all participating Utility Chains, it should be designed as a stable, error-resistant module from the start.

For a little insight:

This transfer pathway can be used for a much wider variety of assets than just the native URI. In future iterations, services such as Escrow, a DEX, loans, etc., can use this same pathway

CoinTransfer module

The CoinTransfer module defines the monetary state machine, not just the message format

ICMS carries the transfer instructions, acknowledgements, and completion commands, but ICMS alone is not sufficient to make native-coin transfer safe. The CoinTransfer module is the chain-resident state machine that determines whether value is reserved, minted, locked, unlocked, revoked, refunded, or finalized. It exists on UriChain and on each Utility Chain and exposes a stable message and event surface so the same protocol can be applied consistently across the network. The source-side action is a reservation of URI on UriChain and the destination-side action is a mint-then-lock of URI-<UtilityChainName> pending source confirmation.

Case 1: UriChain → Utility Chain Transfer (Simple)

In this first case, the source chain is UriChain and the destination is a Utility Chain. There are two methods. The first is a simple transfer using the ICMS. **However, a second method is introduced that utilizes Hashed Time Lock Algorithms (HTL), on top of our existing ICMS infrastructure. When transfer amounts pass a certain threshold, the HTL method may be required for enhanced security.**

Simple Transfer Method Using ICMS

The general flow is:

- Lock or reserve the backing amount on UriChain's CoinTransfer module
- ICMS event is emitted and picked up by the Message Server which then delivers it to the appropriate chain
- Validate the received message from the Message Server. If OK, mint a destination representation on the Utility Chain's CoinTransfer module in a locked state
- Acknowledge that mint preparation back to UriChain through the ICMS
- Let UriChain decide whether the transfer is still valid and timely
- Send either an unlock command or a revoke command back to the Utility Chain.

This two-phase release model prevents the Utility Chain from crediting spendable value until the source side has conclusively decided the transfer should complete.

1. The user submits a transfer request to UriChain, and the request is validated before any cross-chain action is created

A main-to-Utility transfer begins as a normal UriChain transaction submitted by the sender. The transaction targets the CoinTransfer module and specifies the destination Utility Chain, destination receiver address, transfer amount, timeout bounds, and any cross-chain fee or relay-tip fields. The module must validate the request completely before it creates any pending state or emits any outbound ICMS event. That includes amount checks, sender balance checks, destination-chain support checks, address syntax checks, timeout reasonableness, and the current health and emergency status of the module.

2. UriChain reserves the backing URI, creates a transfer record, and emits the outbound Type 1 ICMS request

If the request passes validation, UriChain does not immediately destroy the sender's funds and hope the Utility Chain eventually behaves correctly. Instead, the CoinTransfer module reserves the backing amount in a module-controlled balance, records the transfer in a pending table, collects the cross-chain relay fee and any tip, and emits a committed Type 1 ICMS request whose payload tells the destination Utility Chain to prepare a mint in locked form. This matches the conservative source-side requirement that funds must be recoverable if the transfer later fails or times out.

3. UriChain keeps the transfer pending and supervises timeout before finalization

Once the outbound ICMS event has been emitted, the transfer remains pending on UriChain until it reaches one terminal outcome. A simple but important rule applies here: if the transfer times out before the destination preparation is acknowledged and finalized, the sender's transfer amount must be returned from the module reserve, but chain gas is not refunded. Cross-chain relay fees and tips can follow the policy defined by the module; a conservative initial policy is to refund unused tip but not runtime gas.

4. A message server observes the committed request and transports it as a Type 1 ICMS delivery

The transport stage reuses the same Type 1 ICMS mechanics already established earlier. A message server connected to UriChain listens for committed ICMS events from trusted UriChain endpoints, decodes the COIN_TRANSFER_MAIN_TO_UTILITY_PREPARE_V1 payload, performs transport-level checks, signs the current hop, and forwards the request toward the destination Utility Chain. The server checks metadata such as destination chain, timeout, duplicate request ID, and destination-chain health. It does not re-judge the business validity of the transfer amount or sender balance; those are source-chain responsibilities. If the destination is not directly reachable, the message can be relayed through another message server using the same bounded path metadata and hop controls described earlier.

5. The destination Utility Chain validates the delivery and prepares the mint in locked form

When the destination Utility Chain receives the Type 1 delivery transaction from the message server, it must first authenticate the transport. That means verifying that the delivering message server is authorized for this chain, verifying the delivery signature, checking replay protection, confirming that the message is actually intended for this chain's CoinTransfer module, and confirming that the remaining timeout window is still acceptable. The destination should not accept a request whose deadline is so close that it may expire immediately after mint preparation. A conservative minimum remaining-time policy can be enforced here.

Once the transport checks pass, the Utility Chain does **not** immediately credit the user with a spendable balance. Instead, it mints the destination representation URI-<UtilityChainName>, applies any destination-side inbound fee policy, and places the recipient's net amount into a

locked holding record. This preserves the two-phase completion model: destination value exists, but it is not yet spendable until UriChain later sends an explicit unlock.

6. The destination chain emits a preparation acknowledgement, and the acknowledgement returns to UriChain through the same Type 1 transport pattern

Once the destination mint-lock step succeeds, the destination chain emits a receipt event that describes what it prepared: transfer ID, destination denom, gross minted amount, destination fee, net locked amount, timestamp, and release-policy metadata. A message server attached to the destination Utility Chain observes that committed acknowledgement and returns it to UriChain as a Type 1 acknowledgement flow, just as in the earlier Type 1 section. If the destination rejected the request instead, the destination emits an error event instead of a preparation acknowledgement.

7. UriChain receives the acknowledgement, re-checks timeout and metadata, and decides whether to unlock or revoke

The most important decision in this case happens when UriChain receives the destination acknowledgement. UriChain is the chain that holds the canonical backing URI, so UriChain is the chain that decides whether the transfer should be completed. It verifies the message-server signature, correlates the acknowledgement to the pending transfer_id, checks that the metadata matches the original request, and checks whether the source-side transfer has already timed out. If the transfer is still valid, UriChain commits the transfer and sends an unlock instruction. If the transfer has already timed out or the acknowledgement is malformed, UriChain returns the sender's backing URI and sends a revoke instruction to the destination Utility Chain so the prepared mint can be voided.

8. If UriChain commits the transfer, it sends an unlock command and the Utility Chain releases the locked balance to the receiver

If the transfer is still valid when UriChain processes the acknowledgement, UriChain emits a second Type 1 ICMS request whose payload is CoinTransferUtilityUnlockV1. The message server network carries that unlock request to the destination Utility Chain in the same way as any other Type 1 message. On receipt, the Utility Chain verifies the message-server signature and transfer metadata, finds the matching pending mint-lock record, checks that the net amount and denom match, and then releases the locked balance to the receiver account. If the release policy is IMMEDIATE, the funds can be released immediately. If the release policy is MULTISIG, TIMELOCK, or VESTING, the Utility Chain records the prepared balance under that policy and only transitions it to spendable according to the required conditions.

9. If UriChain cancels the transfer, it sends a revoke command and the Utility Chain voids the prepared mint

If UriChain decides that the transfer must not complete, it sends CoinTransferUtilityRevokeV1 instead of CoinTransferUtilityUnlockV1. The destination Utility Chain then finds the matching pending mint-lock record and cancels it. Because the prepared amount was still under module control, the Utility Chain can burn the receiver's locked amount cleanly rather than clawing back already-spendable funds from a user account. That is one of the main reasons the two-phase release model exists.

10. Large-value transfers can keep the same transport path while using stricter release policies

The transport path does not need to change for large-value transfers, but the monetary release policy may need to become stricter. The same CoinTransferMainToUtilityPrepareV1 request can carry a release policy indicating that the prepared mint will not become spendable immediately upon unlock delivery. Instead, the destination chain can require a committee multisignature, a timelock, a vesting schedule, or a combination of those controls. This allows the system to stay conservative without inventing a second incompatible transport protocol for high-value movement.

11. The protocol is complete only when accounting closes cleanly on both chains

A main-to-Utility native-coin transfer is not finished when the first ICMS request is delivered, and it is not even finished when the Utility Chain prepares the mint. It is finished only when both chains have reached a terminal state that agrees with the supply ledger:

- **Completed:** UriChain still holds the backing URI reserve for that Utility Chain, the destination representation exists, and the user's net amount has been released or scheduled under policy.
- **Revoked:** the source-side backing URI has been returned or remains uncommitted, and the destination prepared mint has been burned or otherwise voided before becoming spendable.
- **Timed out:** UriChain has returned the sender's amount, and the Utility Chain has either never prepared the mint or has already processed a revoke.
- **Failed:** the transfer was rejected before it could enter a valid pending state.

That closure discipline matters because Universal Cross-Chain Payments are only credible if every transfer concludes with one clear monetary outcome and one corresponding ledger state. The network can tolerate relay delay, transient destination failure, and retry behavior in transport. It cannot tolerate ambiguous native-coin accounting.

Visual summary: URI transfer from UriChain to a Utility Chain

```
UriChain / Main                msg-server-01                Utility Chain
-------------------------------------------------------------------------------------------------------
User -> CoinTransfer.request_main_to_utility()
  validate sender, dst chain, dst address, amount, timeout, fees
UriChain CoinTransfer
  reserve backing URI in module
  create transfer record
  emit EventICMType1RequestedV1
  payload=COIN_TRANSFER_MAIN_TO_UTILITY_PREPARE_V1
|--------------- ICMS PREPARE REQUEST -------->|
                    observe committed event
                    validate metadata
                    sign relay
                    forward to destination
                                                 verify message-server signature
                                                 verify chain/module/timeout/replay
                                                 mint URI-UtilityChain to module
```

```
                                                    collect target fee per policy
                                                    lock net amount for receiver
                                                    emit prepared ack receipt
|<-------------- ICMS PREPARED ACK -------------------|
UriChain CoinTransfer
  verify ack metadata
  lookup pending transfer
  check timeout still valid?
  if valid:
    emit COIN_TRANSFER_UTILITY_UNLOCK_V1
  else:
    refund sender on UriChain
    emit COIN_TRANSFER_UTILITY_REVOKE_V1
|--------------- UNLOCK or REVOKE ------------------->|
                                          if UNLOCK:
                                            release locked balance to receiver
                                            or activate release policy
                                          if REVOKE:
                                            burn or void prepared locked mint
                                        close transfer record
```

Hashed TimeLock Contracts (HTLC) Method

In the case that Hash Time Locks are required, the source chain is still UriChain and the destination is still a Utility Chain, but the completion path is now HTLC-based (yet still built on top of the ICMS infrastructure):

- UriChain reserves the backing URI and emits a Type 1 ICMS prepare request.
- the destination Utility Chain mints the destination representation into module custody under a hash lock and expiry,
- the destination acknowledges the prepared contract back to UriChain,
- UriChain activates the contract if the transfer is still timely and valid,
- the receiver claims the destination-side funds by revealing the correct preimage before expiry,
- the destination emits a claim receipt back to UriChain,
- UriChain finalizes the transfer on proof of claim,
- if the claim never occurs before expiry, the contract is revoked or expires and the sender is refunded from source-side backing.

Big-picture flow of a main-chain-to-Utility-Chain URI transfer with HTLC

1. **Transfer request admission on UriChain.** The sender submits a transfer request to the CoinTransfer module on UriChain, specifying the destination Utility Chain, receiver address, amount, timeout fields, and a hash-lock commitment.
2. **Source-side validation and reservation.** UriChain validates the request, reserves the backing URI, collects relay fees and tips, creates a transfer record, and emits a structured outbound Type 1 ICMS prepare request containing the hash lock and claim-expiry metadata.

3. **Source-side pending supervision.** UriChain keeps the transfer in a pending table and monitors both pre-activation timeout and later HTLC expiry conditions.
4. **Message-server observation and transport.** A message server connected to UriChain observes the committed outbound ICMS event, validates transport metadata, signs the relay, and forwards the request toward the destination Utility Chain.
5. **Destination-side transport validation and HTLC preparation.** The destination Utility Chain verifies the delivering message server, checks replay and timeout conditions, mints URI-<UtilityChainName> into module custody, and stores a destination-side HTLC record under the supplied hash lock and expiry.
6. **Preparation acknowledgement back to UriChain.** The destination chain emits a preparation acknowledgement, and a message server returns it to UriChain through the same Type 1 ICMS transport pattern.
7. **UriChain activation decision.** UriChain receives the acknowledgement, verifies it, and if the transfer is still valid, sends an activation instruction to the destination Utility Chain. If not, it refunds the sender and sends a revoke instruction instead.
8. **Receiver-side HTLC claim.** Once activated, the receiver on the Utility Chain submits the preimage to claim the locked amount before expiry. The destination chain verifies the hash, applies any claim-time destination fee, releases the net amount, and records the claim.
9. **Claim acknowledgement and source finalization.** The destination chain emits a claim acknowledgement, and UriChain finalizes the source-side transfer record once it receives and verifies that acknowledgement.
10. **Expiry, revoke, and refund handling.** If no valid claim occurs before expiry, the destination-side HTLC is revoked or expired, the prepared mint is burned or voided, and the sender is refunded from UriChain's reserved backing.

Visual summary: URI transfer from UriChain to a Utility Chain with HTLC

```
UriChain / Main                    msg-server-01                    Utility Chain

-------------------------------------------------------------------------------------------------------------------------

User -> CoinTransfer.request_main_to_utility_htlc()

  validate sender, dst chain, dst address, amount, timeouts, hash_lock

UriChain CoinTransfer

  reserve backing URI in module

  create transfer record

  emit EventICMType1RequestedV1

  payload=COIN_TRANSFER_MAIN_TO_UTILITY_HTLC_PREPARE_V1

|---------------- ICMS HTLC PREPARE REQUEST ----------->|

                        observe committed event

                        validate metadata

                        sign relay

                        forward to destination

                                                verify message-server signature

                                                verify chain/module/timeout/replay

                                                mint URI-UtilityChain to module

                                                create PREPARED HTLC under hash_lock

                                                emit prepared ack

|<--------------- ICMS HTLC PREPARED ACK -------------|

UriChain CoinTransfer

  verify ack metadata

  lookup pending transfer
```

```
  check prepare deadline still valid?
  if valid: emit COIN_TRANSFER_UTILITY_HTLC_ACTIVATE_V1
  else:  refund sender on UriChain
    emit COIN_TRANSFER_UTILITY_HTLC_REVOKE_V1
|---------------- HTLC ACTIVATE or REVOKE ------------------------------>|
                                                        if ACTIVATE:
                                                          mark HTLC ACTIVE
                                                          wait for receiver preimage claim
                                                        if REVOKE:
                                                          burn prepared mint
                                                          close contract
                                                       Receiver -> claim_htlc(preimage)
                                                        verify sha256(preimage)==hash_lock
                                                        verify expiry not reached
                                                        release net amount to receiver
                                                        emit claimed ack with preimage
|<--------------- ICMS HTLC CLAIMED ACK —------------------------------|
UriChain CoinTransfer
  verify claim ack and preimage
  finalize source transfer as COMPLETED
  backing URI remains reserved on main chain
If no claim before expiry:
  destination expires / burns HTLC
  source refunds sender after grace window
  source sends revoke if cleanup still needed
```

Case 2: Utility Chain → UriChain Transfer

Case 2 follows an analogous workflow in the reverse direction, with burns instead of mints on the Utility Chain's CoinTransfer module. Upon confirmation, the CoinTransfer module on UriChain will send the URI to the receiver's account on UriChain.

Once again, based on the sending amount or other situations, usage of the Hash Time Lock method may be mandated to ensure security.

Case 3: Utility Chain A → Utility Chain B

Case 3 works through a combination of Case 2 followed by Case 1. More efficient methods are currently under development and the whitepaper may be updated to reflect it in future revisions.

Failure handling and safety controls

Every transfer record must support timeout, revoke, duplicate suppression, and emergency pause. Message servers must refuse forwarding when metadata is malformed, the destination Utility Chain is unhealthy, or the hop limit has been exceeded. Large transfers may require additional multisignatures from validators of the chains involved before UNLOCK or final settlement. In some accounts or risk classes, destination funds may remain time-locked, vesting-locked, or multisig-locked even after receipt. If future infrastructure reaches effectively guaranteed delivery, Method 1 can be simplified, but the conservative two-step flow is the safer default for a currency system.

Economics (Tokenomics)

Summary/At a Glance

URI token is the standard unit of utility across UriNet—powering access, staking/governance, payments, ICMS-based coordination, and participation in shared network services and modules.

Monetary policy follows a hybrid model:

- **Genesis supply:** 1,000,000,000 URI.
- **Denominations:** 1 URI = 10^9 Kwak (KWAK), the minimum sub-denomination (10^-9).
- **Inflation bounds:** Chain-configurable floor and ceiling with initial parameters set to a **minimum of 1%** and a **maximum of 10%** per year.
- **Transaction fees (base + priority) and burning:** Every transaction pays a **base fee** (per signature). Today, **50% of the base fee is burned** and **50% goes to the current block leader (proposer).** Users can optionally add **priority fees** (aka compute-unit price) to get included faster. **100% of priority fees go to the block leader**.
- **Automatic control (default):** Inflation adjusts each block toward a **target staking ratio of 67%**. If stake is below target, inflation moves up to incentivize staking; if above, it moves down.
- **Manual override (emergency/exceptional):** Governance can set inflation directly when warranted, subject to on-chain quorum and supermajority (see Manual Override).
- **Reward routing:** A dedicated minting module mints block rewards and hands them to the distributor module, which deducts any community tax and proposer reward (if any), then

pays validators pro-rata by voting power; delegators receive rewards net of validator commission.

- **Governance agility:** Monetary parameters are module settings; approved changes take effect immediately after the voting period and are applied starting with the very next block.

Token Utility

- **Security:** Stake or collateralize URI to participate, directly or via delegation, in UriChain consensus and in shared network services that require URI-backed economic commitment.
- **Fees and service access:** URI is the common utility asset used to pay for execution, ICMS messaging, multichain workflow coordination, settlement, and shared-service access across UriNet, including both UriChain and Utility Chains.
- **Incentives:** Validators and delegators earn URI emissions and fee-based rewards; proposers receive any chain-configured proposer bonus. URI-denominated network usage also reinforces a common economic layer across otherwise separate chains and services.

Monetary Policy

1) Automatic Inflation Adjustment (default)

Objective: Maintain a healthy security budget while minimizing dilution by steering the network toward a target staked ratio.

Core parameters (governance-set):

- goal_staked: target staking ratio; initially 0.67 (two-thirds).
- inflation_min, inflation_max: hard bounds; initially 1% and 10% annualized.
- inflation_rate_change: maximum annual rate at which inflation can move toward the target (applied incrementally per block).
- blocks_per_year: cadence used for pro-rata minting; with 0.5s blocks, this is 2 × 86,400 × 365 = 63,072,000.
- mint_denom: base denomination for minting and accounting (KWAK).

Per-block mechanism:

1. **Measure staking:**

   staked_ratio = staked / total_supply

- **Update inflation (bounded):**

  inflation_rate_change_per_year = (1 - staked_ratio / goal_staked) * inflation_rate_change

  inflation_next = clamp(inflation_current + inflation_rate_change_per_year / blocks_per_year, inflation_min, inflation_max)

2. Intuition: if staked_ratio < goal_staked, the term is positive and inflation rises; if above, it falls.
3. **Compute provisions:**

   annual_provisions = inflation_next * total_supply

   block_mint = annual_provisions / blocks_per_year
4. **Mint & route:**

   The mint module creates block_mint (in KWAK) and sends it to the fee collector.
5. **Distribute at block start:**

Example (illustrative):

Assume goal_staked = 0.67, inflation_rate_change = 0.10 (10%/yr slope), inflation_current = 0.035 (3.5%), and staked_ratio = 0.50.

Then 1 - 0.50 / 0.67 ≈ 0.254; per-year change ≈ 0.0254. Per-block delta is ≈ 0.0254 / 63,072,000 ≈ 4.03×10^-10. Inflation ticks up by that amount this block (bounded by 1–10%).

This slow, continuous controller dampens volatility and makes the security budget adaptive without governance micromanagement.

2) Manual Override (exceptional governance)

When urgent conditions or clear public-interest reasons exist (e.g., acute security budget gaps, systemic shocks), governance can **temporarily set community tax rate and/or inflation directly** or adjust parameters outside the normal cadence.

- **Quorum:** At least **50% of staked weight** participates in the vote.

- **Supermajority:** Among participants, at least **two-thirds (⅔)** must vote to approve.
- **Effectivity:** Upon approval, the new rate/parameters replace the old ones and take effect **immediately after the voting period**; the **very next block** uses them.
- **Scope:** Changes may set a fixed inflation for a defined period or update controller parameters (goal_staked, bounds, slope).
- **Reversion:** Unless otherwise specified, the system reverts to the automatic controller after the override window elapses.

Implementation detail: modules expose an authority address (typically the governance module/DAO). A successful parameter-change proposal updates the live parameters atomically.

Emissions and Fee Distribution

Newly minted tokens and collected fees are routed as follows:

1. **Community tax (optional):** If enabled, a small percentage of each block's rewards/fees is diverted to the **community pool** for grants, maintenance, and ecosystem programs (rate and uses are governance-configured).
2. **Proposer reward: 0% (default, block reward). However,** DAO can vote to set **1–5%** proposer bonus (chain-configurable) to incentivize low-latency, high-availability block production.
3. **Security rewards:** The remainder is distributed **pro-rata by voting power** across **staked validators**; delegators receive their share net of the validator's commission.

4. **Transaction fees (base + priority) and burning:** Every transaction pays a **base fee** (per signature). Today, **50% of the base fee is burned** and **50% goes to the current block leader (proposer)**. Users can optionally add **priority fees** (aka compute-unit price) to get included faster. **100% of priority fees go to the block leader**.

Rewards accrue continuously and can be claimed by recipients on demand.

Governance and Parameter Management

Inflation behavior is entirely **parameterized**. Governance can update:

- Controller target and slope: goal_staked, inflation_rate_change
- Bounds: inflation_min, inflation_max
- Cadence and denom: blocks_per_year, mint_denom
- Distribution knobs: community tax rate, proposer reward rate, transaction fee burning ratio, etc.

A successful parameter-change proposal updates the active configuration; changes **apply beginning with the next block** following the voting period. This gives UriNet both **predictability** (automatic control) and **optionality** (fast-acting override) without protocol upgrades.

Supply, Units, and Accounting

- **Genesis supply:** 1,000,000,000 URI allocated per the genesis distribution plan.
- **Units:** 1 URI = 1,000,000,000 KWAK; all minting and settlement occur in KWAK to avoid rounding.
- **Total supply accounting:** Updated each block as emissions are minted and burned (if/when fee burns or clawbacks are introduced by governance in the future).

Design Rationale

- **Security-first:** A target staked ratio of ~⅔ balances decentralization, economic security, and liquidity for real-world utility.

- **Predictable yet responsive:** Per-block, bounded adjustments avoid fee and reward whiplash while still reacting to sustained shifts in stake.

- **Clear roles and incentives:** Proposer bonuses and optional community tax ensure operational excellence and long-term commons funding without diluting validator/delegator alignment.

- **Governance agility:** Immediate, block-level effectivity of approved parameter changes enables rapid crisis response without hard forks.

At-a-Glance Parameters (initial, governance-tunable)

- goal_staked: 0.67
- inflation_min: 0.01
- inflation_max: 0.10
- inflation_rate_change: (set by governance; defines the controller's annual slope)
- blocks_per_year: 63,072,000 (0.5s blocks)
- proposer_reward: 0%–5% of block rewards/fees (0% default)
- Fee_burting_ratio: 0.5 (50% of the base fee is burned)
- community_tax: 0–X% (optional; to the community pool)
- mint_denom: KWAK (10^-9 URI)

Worked Example (end-to-end, illustrative)

- Inputs at block N:

  total_supply = 1,000,000,000 URI

  staked = 600,000,000 URI → staked_ratio = 0.60

  goal_staked = 0.67, inflation_current = 0.03, inflation_rate_change = 0.10,

  blocks_per_year = 63,072,000

- Controller update:

  1 - 0.60/0.67 ≈ 0.10597 → per-year change ≈ 0.01060

  Per-block delta ≈ 1.68×10^-10 → inflation_next ≈ 0.030000000168 (bounded in [1%,

10%])

- Provisions:

  Annual ≈ 0.030000000168 × 1,000,000,000 = 30,000,000.168 URI

  Per-block ≈ 30,000,000.168 / 63,072,000 ≈ 0.4758 URI

- Distribution (example policy):

  Community tax 2% (if any) → 0.00952 URI to pool

  Proposer reward 2% of remainder (if any) → ~0.00930 URI to proposer

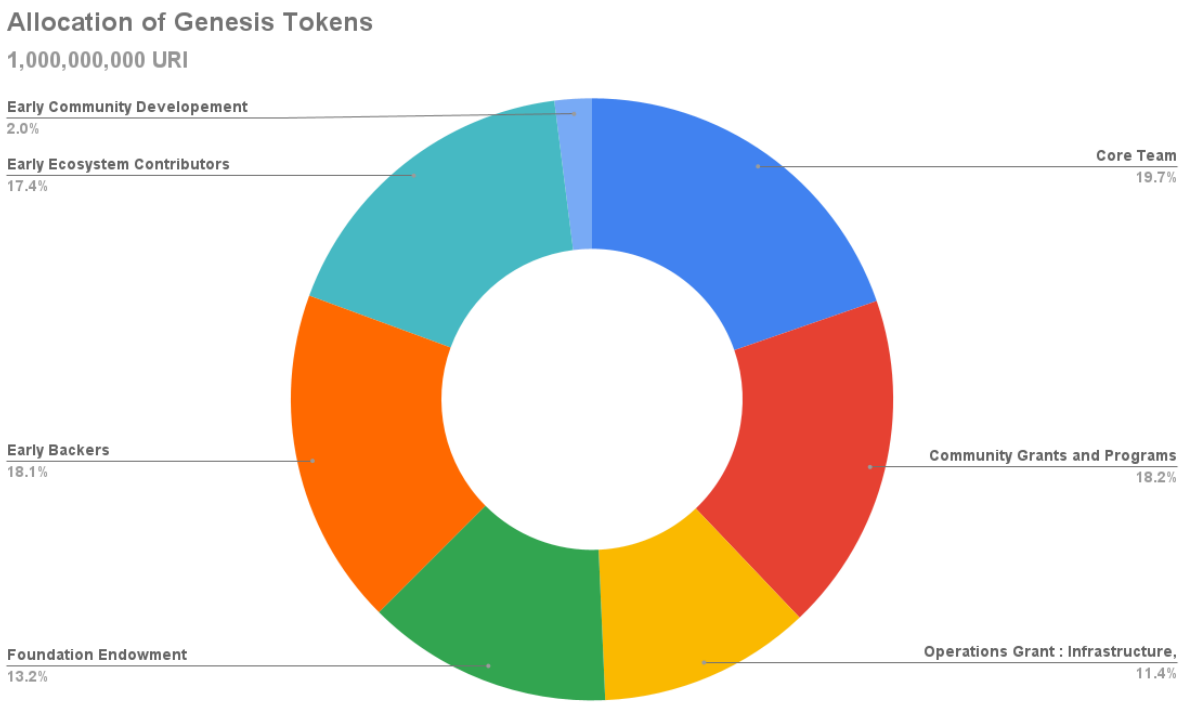
  Remainder → pro-rata to validators; delegators receive net of commission.

This example illustrates the precise, incremental behavior of the controller and the reward routing per block.

By combining a **self-stabilizing controller** with **clear distribution mechanics** and **fast governance levers**, UriNet's economics prioritize durable security, predictable developer/user experience, and the long-term health of the ecosystem.

Overall Allocation and Supply Schedule

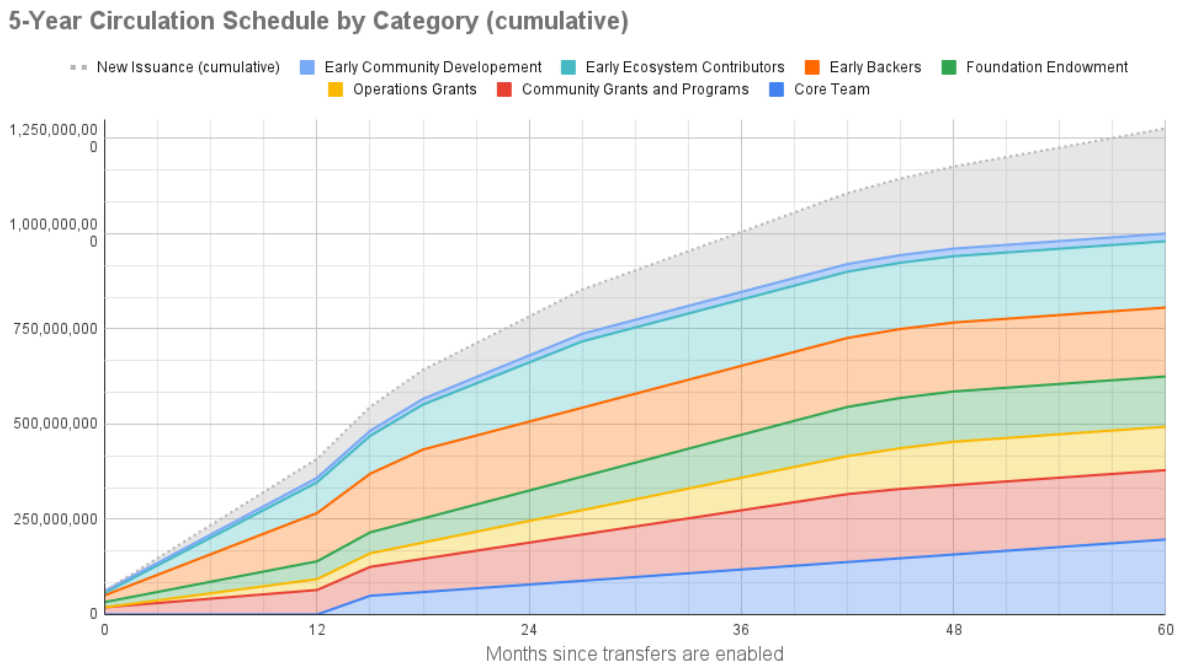
There will be 1,000,000,000 URI tokens created at Genesis. This will be allocated in the following way, where each category is described in greater detail in the following sections:



> "Circulating Supply" is every token that isn't subject to a lockup at a particular time. The vast majority of the initial supply of URI tokens is subject to long-term lockups.

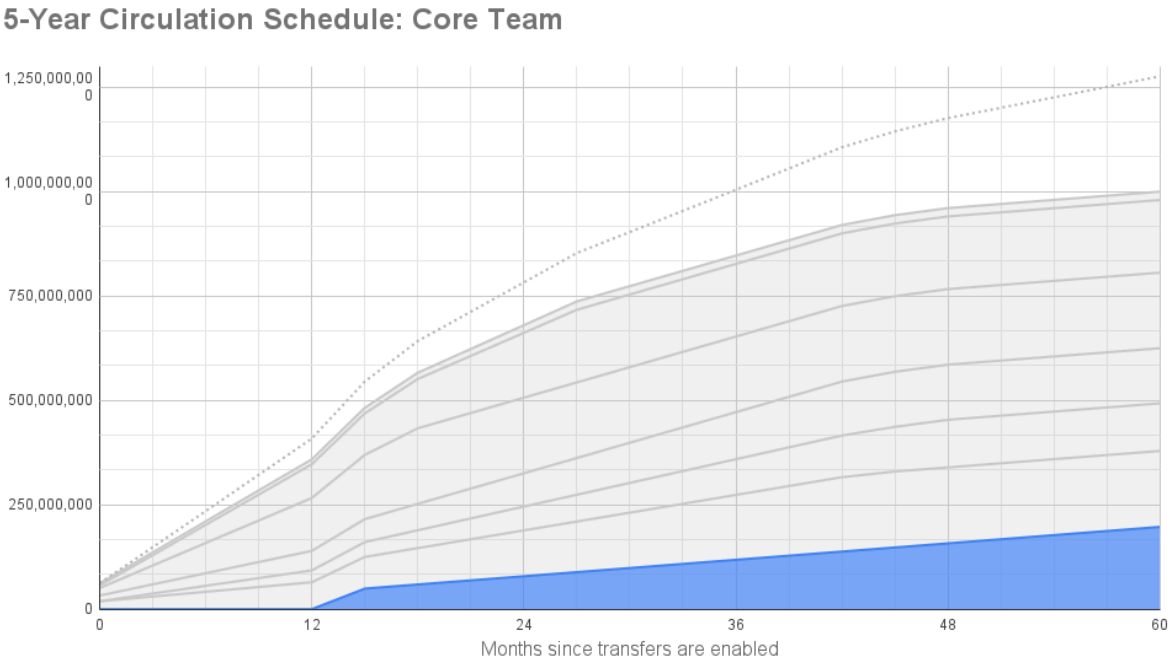
The chart below shows the increase in circulating supply as tokens unlock and new tokens are minted. The dotted line shows the total supply as it increases due to new

issuance (but ignores any burning that would be due to fees). This and all charts will begin on the date that transfer restrictions are lifted by Governance/DAO/Foundation.



Core Team

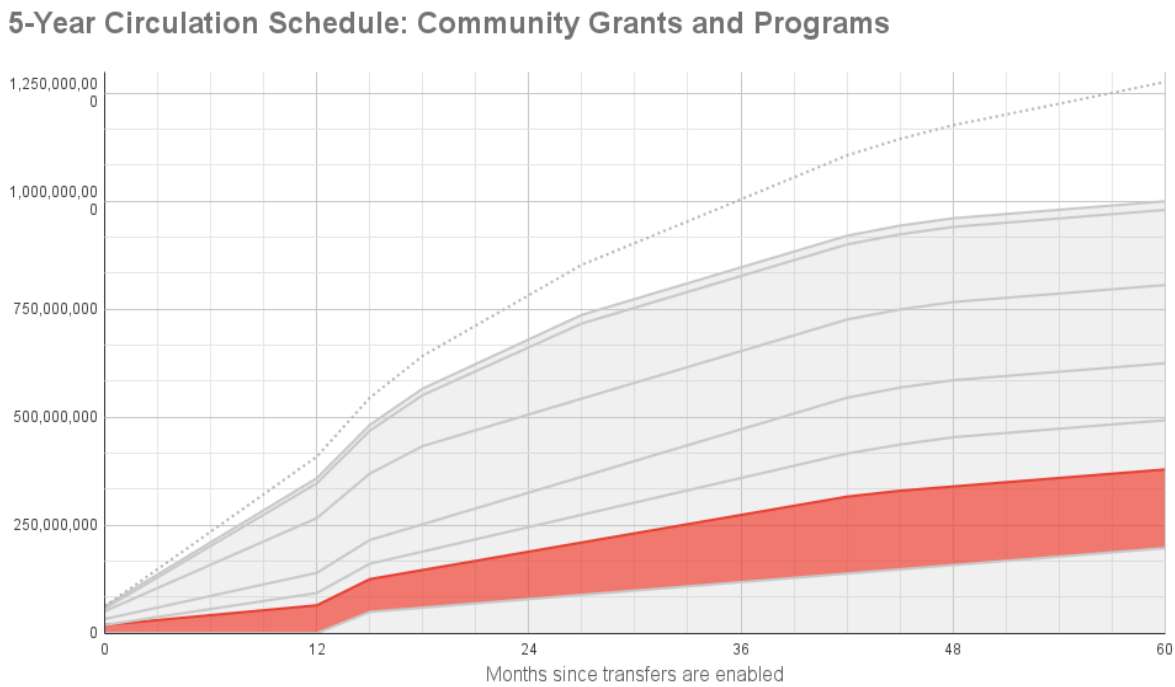
This allocation recognizes the people who turned the idea into running code and who will continue maintaining it after launch. It covers founding engineers, product and protocol researchers, security leads, and key advisors whose work spans multiple releases. Tokens here are long-term oriented: subject to multi-year vesting with an initial cliff, aligning incentives with delivery of roadmap milestones rather than short-term market moves. A portion is reserved for future strategic hires so we can recruit top talent without repeatedly tapping the treasury. Day-one liquidity is intentionally limited to keep this bucket aligned with network health and long-run value creation.



Community Grants and Programs

This pool funds public-goods style work that grows the pie for everyone: protocol research, developer tooling, libraries and SDKs, documentation, education, hackathons, and community-run initiatives. Grants are milestone-based and evaluated by an open process with clear criteria; larger, multi-quarter efforts may be staged with tranches and progress reviews. We expect most outputs to be open-source and permissively licensed so they’re reusable across the

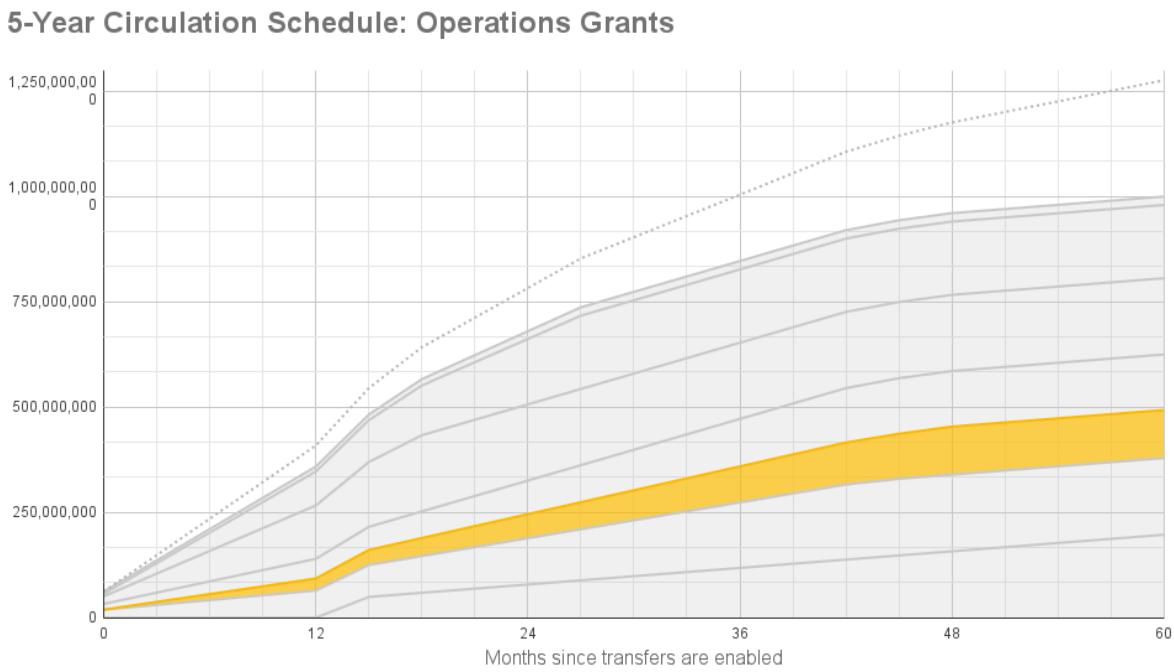
ecosystem. Distribution is paced over time to avoid shocks and to ensure we can support promising new work as real-world needs emerge.



Operations Grant : Infrastructure, Operation cost, etc...

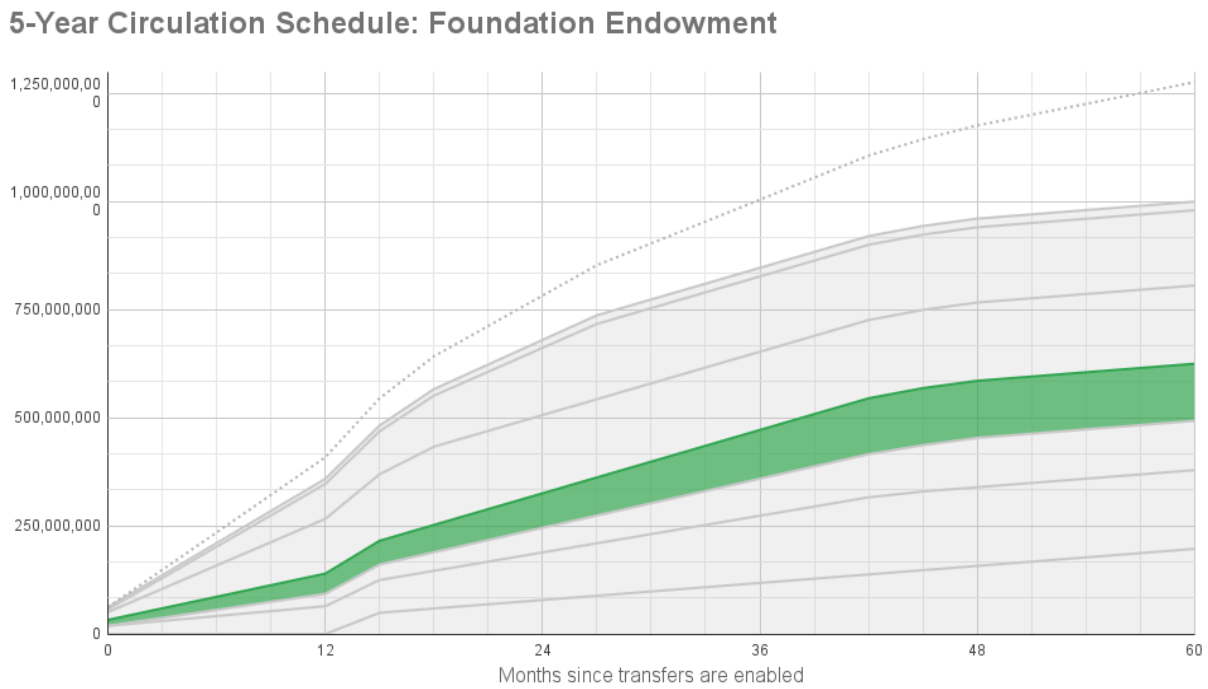
Launching and running a network takes reliable, boring excellence. This bucket covers essential operating expenses that don't fit cleanly into "grants" or "team" compensation: security audits and bug bounties, observability and data infra, seed validator bootstrapping or testnet

incentives, wallets/explorer maintenance, developer support, legal/compliance and statutory filings, insurance, incident response, and vendor costs (hosting, CI/CD, etc.). Where practical, payouts are denominated in stable assets or scheduled to minimize market impact. The goal is predictable, transparent funding of the “keep-the-lights-on” work that underwrites user trust.



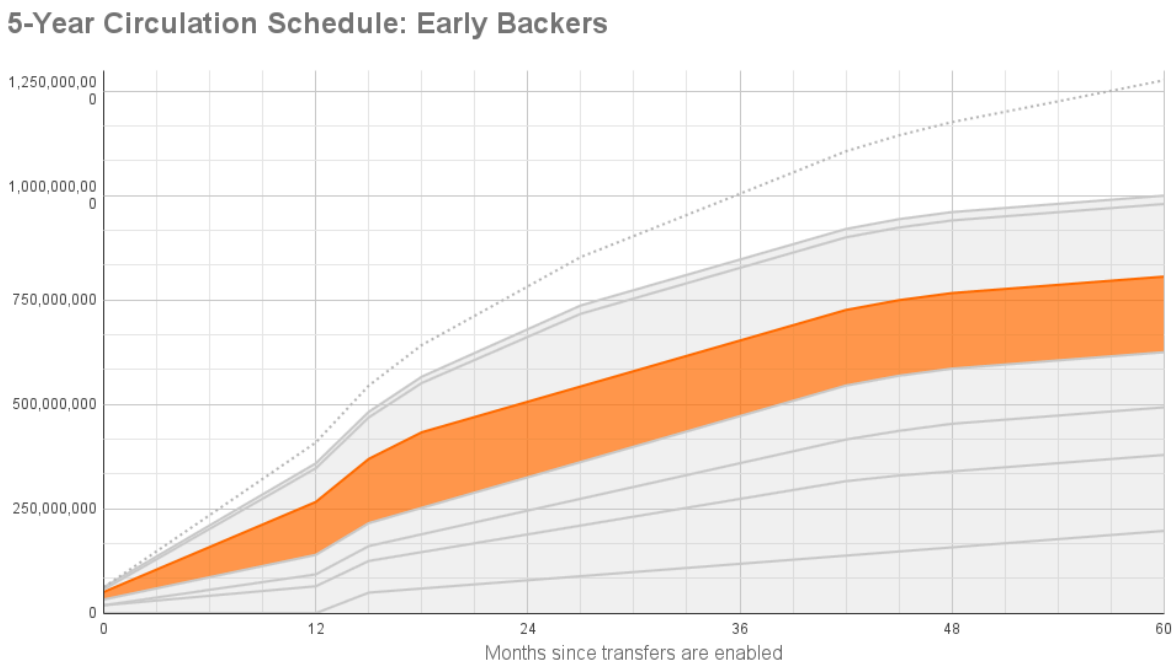
Foundation Endowment

The endowment is the long-horizon backbone of ecosystem stewardship. It sits with the independent Foundation and is governed by a published spending policy that favors stability over cycles. Proceeds are used to fund non-commercial public goods, core protocol initiatives, academic/standards collaborations, and emergency support during downturns. Custody, reporting, and conflict-of-interest rules are conservative by design, with periodic transparency reports on holdings, performance, and disbursements. This helps ensure the project can serve builders and users for years, not just a market season. (NEAR framed a similar bucket to provide durable, neutral funding for stewardship.



Early Backers

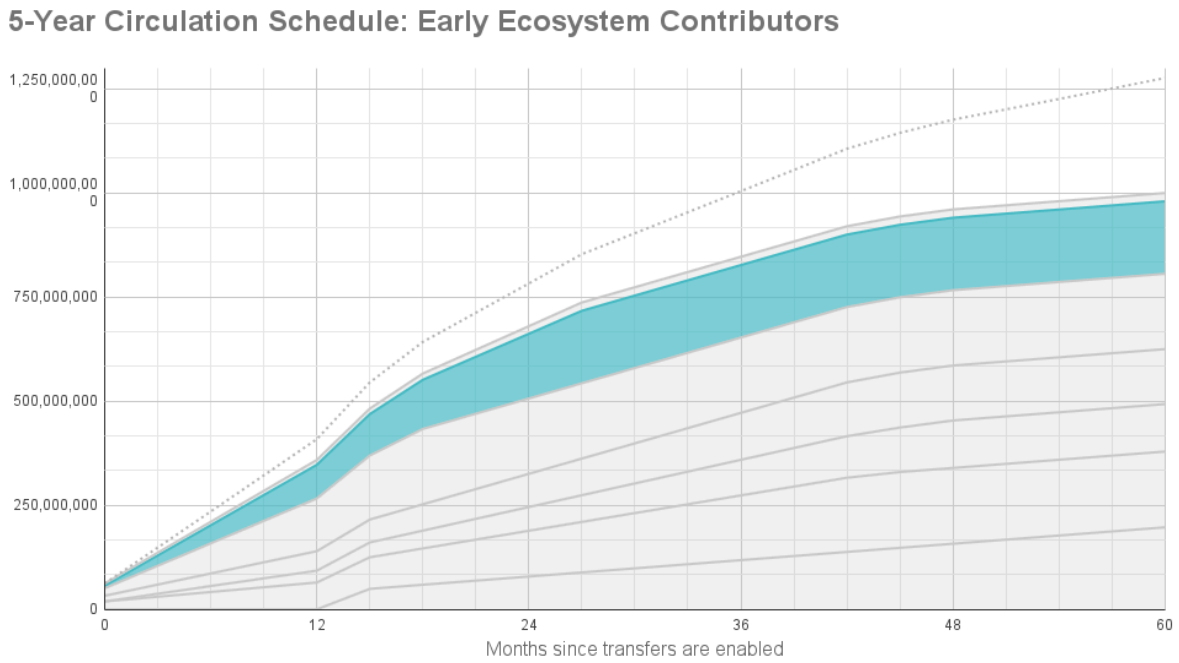
Early backers supplied the risk capital that made multi-year R&D and early ecosystem formation possible before there was usage or revenue. This allocation compensates that risk while aligning incentives with long-term network success. Tokens are subject to standard lockups (cliffs followed by linear vesting), transfer restrictions during sensitive phases, and clear disclosures. Participation typically includes value-add expectations—recruiting support, market development, technical advisory, and introductions that accelerate adoption—so this bucket is about more than capital alone. (In NEAR's framing, this corresponded to Backers/Small Backers with tiered schedules.



Early Ecosystem Contributors (Developers + Utility Service Partners)

This pool rewards the folks who show up early and ship. It includes teams that launched foundational applications or utility services (payments, identity/compliance, oracles, data, comms), operators who supported incentivized testnets or critical community tools, and partners who onboarded real users or enterprises. Distributions are milestone-based and may include performance-linked vesting (for example, tied to uptime, usage thresholds, or open-sourcing commitments). The aim is to recognize concrete contributions that made the

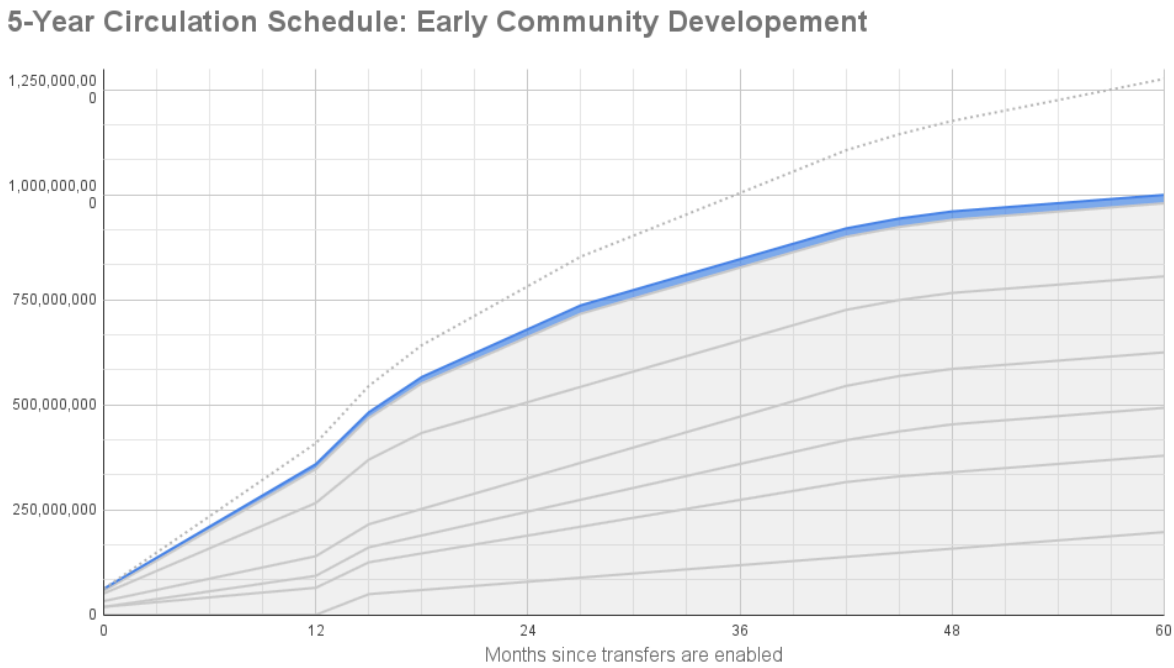
network useful from day one and to keep these builders invested in its growth. (Comparable to NEAR's Early Ecosystem bucket, which explicitly covered beta-program builders, analytics, and testnet support.



Early Community Development

This pool jump-starts the first wave of real users and contributors. It covers things such as airdrops, newcomer onboarding, simple quests, local meetups, translations, and community-run

tools/moderation. The goal is straightforward: seed healthy network effects and give early ownership to the people who show up and stick around.



Real-World Use Cases

UriNet is designed for everyday utility: each product runs on its own sovereign Utility Chain, natively connected to others through UriChain and UriPortal. Below we outline concrete use cases that map directly to the modules and interchain messaging described earlier, consolidating every item in our internal notes into coherent, no-fluff application categories.

Example Applications

1) Finance & Digital Markets

RWA/STO issuance and servicing. Launch compliant, single-purpose chains for asset tokenization (real estate, revenue-share notes, invoices), with on-chain cap tables, distributions, and transfer restrictions. Identity/compliance modules gate access; universal payments disburse coupons/dividends; a built-in DEX provides primary/secondary liquidity via interchain order routing. Includes support for micro-lender services as a pre-built vertical chain.

NFTs and digital goods. Per-product chains mint and trade NFTs tied to utility (tickets, memberships, game assets). Native interchain messaging lets assets move between games/apps while enforcing royalties and usage policies at the chain level.

Exchange and settlement. UriNet's built-in DEX plus native messaging enables orderbooks, RFQ, or AMM models across chains—settlement occurs atomically network-wide without brittle bridges.

Payments & benefits. Issue programmable benefits, rebates, and loyalty points; settle across chains with universal payments.

Insurance & hedging. Parametric products trigger from oracle proofs (weather, shipping events, price feeds). Users can hedge risks or take positions via a network-wide prediction/derivatives chain.

Research & prediction. A dedicated prediction-market chain offers polls, forecasting, and hedging instruments; results can inform payouts or governance on other chains.

2) Trustless Private Contracts & Escrow

Spin up escrow/trust chains for bilateral or multiparty deals (marketplaces, B2B trade, creator collaborations). Templates include milestone escrow, claim-based insurance, and censorship-resistant agreements enforced by consensus and oracle proofs.

Oracle-triggered execution. Contracts finalize or release funds based on real-world facts supplied by UriNet's oracle chains, removing manual reconciliation and disputes.

3) Social, Media, and Advertising

User-owned social graphs. Community chains manage identity, social relationships, moderation policies, and content storage pointers. Users actually own their handles/content; portability across apps comes by default.

Ads monetization (on/offline). Dedicated ad chains clear impressions, clicks, and conversions with verifiable oracles. Revenue splits execute automatically among creators, publishers, and advertisers; works for online and offline activations (e.g., QR-in-store).

News networks: triple-win model. A news-vertical chain coordinates readers, advertisers, and publishers via transparent metrics, pay-per-article micro-payments, and ad revenue sharing—aligning incentives and reducing fraud.

4) Philanthropy, Public Goods, and Civic Tech

Transparent donations. Charity chains provide end-to-end traceability from donor to program to outcome; programmatic disbursements tie to oracle-verified milestones.

Public-sector tooling. Municipal or agency chains publish budgets, RFPs, and spend in near-real time; citizens audit flows and outcomes. Supports private, trusted deployments where sensitive data isolation is required.

5) Gaming and Interactive Apps

Game-specific chains. Each title (or studio) gets its own app chain for low-latency gameplay, item minting, tournament logic, and marketplace rules—interoperable by default with other titles. Prediction and poll features can be composed for esports formats.

6) Oracles and Real-World Data

Oracle utility chains. UriNet includes native oracle chains that feed prices, events, attestations, and IoT signals to all other chains. Contracts subscribe to these feeds, enabling autonomous execution in finance, insurance, supply chain, and civic apps.

Oracle auction. A marketplace chain lets apps request data with explicit SLAs; oracle operators bid to fulfill requests, staking collateral against accuracy and timeliness. This commoditizes reliable data and reduces costs network-wide.

7) AR/VR and the Uriverse

Uriverse multiverse. AR/VR-first chains anchor identity, assets, and economic rules for virtual spaces that interact with the physical world. Ads, news, prediction games, and insurance can plug in as composable services within immersive environments.

Governance

Governance in UriNet spans four areas:

1. how the core protocol changes
2. how shared resources are allocated
3. how the network's **Utility Chains** (Foundation-run essential/service chains and general purpose chains) are overseen
4. how **ICMS, UriPortal, MSLS, and other network-wide services** are specified and updated

UriChain (the mainnet) provides the on-chain governance rails and treasury; Utility Chains govern themselves locally but agree to baseline interop rules to access shared services through UriPortal.

What gets governed where

- **UriChain (network level).** On-chain votes decide protocol upgrades, network parameters, treasury allocations, and UriPortal/Registry/Services-API changes. UriChain coordinates settlement, payments, and other inter-chain activity. Initially, the Foundation operates community-critical essential/service chains (e.g., security, oracle, messaging, social) and

coordinates their upgrades, support, and grants. Over time, mandates, budgets, and upgrades for these chains are set by network votes.

- **Utility Chains.** Each Utility Chain sets its own application logic, tokenomics (if applicable), and validator policy. Network governance never rewrites a Utility Chain's state; it governs ecosystem privileges: participation in the ICMS/MSLS, oracle access, etc., and compatibility labels exposed via UriPortal.

- **ICMS, UriPortal, and other inter-chain networking infrastructure governance.** The community votes on protocols, parameters, and APIs—so interop remains predictable as UriNet grows.

How decisions happen

- **Discuss off-chain, decide on-chain.** Ideas are debated in public forums, then submitted as proposals with clear, executable payloads. Validators, delegators, and registered participants vote on chain; deposits deter spam.

- **Simple proposal classes.**
    1. Core protocol upgrades and parameter changes on UriChain
    2. Treasury budgets and time-bound grants
    3. Registry/standards and interchain networking infrastructure changes

4. Core Utility Chain upgrade coordination

- **Upgrade rollout.** Each approved protocol release passes through public testing, then schedules mainnet activation at a specific height/epoch once readiness signals are met.

Resource governance (Treasury)

Network revenues (e.g., URI-denominated network/messaging fees, penalties, etc.) fund the commons:

- **Essential/Service Chains:** baseline funding to keep shared services reliable
- **Grants & R&D:** milestone-based builder grants with clawbacks for non-delivery
- **Operations & Infra:** audits, monitoring, incident response, tooling
- **Interchain Networking Infrastructure Maintenance:** Registry operations, API evolution, documentation

Disbursements are transparent and streamed/vested; progress reviews can pause or revoke streams.

Safety & emergencies

For rare, high-risk incidents (critical bugs, cross-chain failures), governance can authorize a narrowly scoped, time-limited emergency action (e.g., temporary quarantine of a misbehaving chain or pause of a vulnerable parameter). Every action includes a short challenge window, automatic expiry unless ratified, and a mandatory post-mortem.

Bootstrap → maturation

At launch, the Foundation coordinates UriChain, essential utility chain operations, and maintains interchain networking's rails. As participation scales, authority migrates fully on-chain: the community elects or renews maintainers, sets budgets, ratifies standards, and directs the UCM within defined guardrails.